

Multimedia

[Appunti di Piai Luca]

Academic Year 2023-2024

Indice

1	La digitalizzazione dei segnali multimediali	5
1.1	I segnali multimediali	5
1.1.1	Sorgenti statiche e dinamiche	5
1.1.2	Categorie dei servizi multimediali	6
1.2	La creazione dei segnali multimediali: ADC	6
1.2.1	Campionamento	7
1.2.2	Quantizzazione	9
1.3	Qualità di un segnale multimediale	11
1.4	Bit rate del segnale multimediale	12
1.5	La ricezione del segnale: DAC	12
1.5.1	L'interpolazione	12
2	Servizi di rete per le applicazioni multimediali	16
2.1	Architettura di rete a strati	16
2.2	Il livello trasporto e le applicazioni multimediali	17
2.2.1	Requisiti di un'applicazione multimediale	17
3	La digitalizzazione di audio, video e immagini	18
3.1	La percezione del suono	18
3.1.1	Codifica percettuale	19
3.2	La digitalizzazione dei segnali audio	19
3.2.1	Digitalizzazione tramite Pulse Code Modulation (PCM): segnali vocali	19
3.2.2	Digitalizzazione della musica	20
3.3	Formati d'immagine e video non compressi	20
3.3.1	Campionamento spaziale, quantizzazione e HDR	20
3.3.2	Immagini a colori	21
3.3.3	Cambiamento di spazio: spazio RGB e YCbCr	22
3.4	Formati video	24
4	Codifica del segnale vocale	25
5	Codifica lossless	26
5.1	Nozioni di base	26
5.1.1	Sorgente, alfabeto, codice e codeword	26
5.1.2	Codici a lunghezza variabile e lunghezza media di un codice	26
5.1.3	Codici a prefisso	26
5.1.4	Codice ottimo	27
5.2	Cenni di teoria dell'informazione	27
5.2.1	Entropia di una variabile aleatoria	27
5.2.2	Teorema di Shannon sulla codifica di sorgente	27
5.3	Codifica di Huffman	27

5.4	La codifica a blocchi	28
5.5	Codifica aritmetica	28
5.6	Tipi di codifica lossless comunemente utilizzati	28
5.6.1	La codifica Exp-Golomb	28
5.6.2	Codifica per categorie e ampiezza	29
5.6.3	Codifica con dizionario	30
5.6.4	Codifica lossless predittiva	30
6	La compressione d'immagini e lo standard JPEG	31
6.1	Introduzione	31
6.2	La trasformata coseno discreta	31
6.2.1	Trasformate lineari ortogonali	31
6.2.2	Scelta della trasformata	32
6.3	JPEG baseline	35
6.3.1	Lo schema di codifica JPEG	35
6.3.2	Quantizzazione in JPEG	35
6.3.3	Codifica lossless coefficiente DC: codifica per categoria ed ampiezza .	37
6.3.4	Codifica lossless a coefficienti AC	37
7	La compressione video: metodologie e standard	40
7.1	Principi di base	40
7.2	La predizione temporale tramite stima e compensazione del movimento . . .	40
7.2.1	Parametri di progetto della stima del movimento	41
7.2.2	Considerazioni sulla stima del movimento	43
7.3	I tipi di immagine ed il group of picture (GOP)	43
7.3.1	I tipi di immagine	43
7.3.2	Il Group Of Pictures	43
8	Indici di prestazione di rete	45
8.1	Principi di base	45
8.1.1	Il servizio di trasporto	45
8.1.2	La pacchettizzazione	45
8.1.3	Paradigma di trasmissione	46
8.1.4	Indici di prestazione di rete	46
8.2	Metriche per il traffico	47
8.2.1	Bitrate, larghezza di banda (bandwidth), Throughput, Goodput . . .	47
8.2.2	Throughput e Goodput	47
8.2.3	PDU, SDU, headers	48
8.3	Metriche di ritardo	48
8.3.1	Ritardo end-to-end, ritardo point-to-point e RTT	48
8.3.2	Ritardo nodale	49

Capitolo 1

La digitalizzazione dei segnali multimediali

1.1 I segnali multimediali

Le sorgenti d'informazione multimediale sono dei **segnali**. I segnali possono essere funzioni di variabile $t \in \mathbb{R}$ (**segnali a tempo continuo**) oppure di variabile $n \in \mathbb{Z}$ (**segnali a tempo discreto**). Allo stesso modo, l'ampiezza di un segnale può variare in un intervallo di $\mathbb{R}$ o in un insieme discreto.

I segnali multimediali possono essere divisi in:

- **Segnali analogici**: sono segnali a tempo continuo ed ampiezza continua

$$s : t \in \mathbb{R} \rightarrow \mathbb{R}.$$

Alcuni esempi di questi segnali sono: pressione acustica, tensione in un circuito elettrico, temperatura istantanea, pressione atmosferica, ecc.

- **Segnali campionati**: sono segnali a tempo discreto ed ampiezza continua

$$s_c : n \in \mathbb{Z} \rightarrow \mathbb{R}.$$

Per esempio: temperatura media giornaliera.

- **Segnali numerici o digitali**: sono segnali a tempo discreto ed ampiezza discreta

$$s_d : n \in \mathbb{Z} \rightarrow \mathbb{Z}.$$

Esistono anche segnali che dipendono da più variabili. Per esempio:

- Immagini analogiche B/N: $I : (x, y) \in \mathbb{R}^2 \rightarrow \mathbb{R}$.
- Immagini digitali B/N: $I : (n, m) \in \{1, \dots, N\} \times \{1, \dots, M\} \rightarrow \{0, 1, \dots, 2^b - 1\}$.
- Immagini digitali a colori: $I : (n, m) \in \{1, \dots, N\} \times \{1, \dots, M\} \rightarrow \{0, 1, \dots, 2^b - 1\}^3$.

1.1.1 Sorgenti statiche e dinamiche

Le sorgenti multimediali possono essere classificate come **statiche** (es:fotografia, testo) o **dinamiche** (es:video, musica, parlato) a seconda che il segnale prodotto dipenda dal tempo oppure no.

1.1.2 Categorie dei servizi multimediali

I servizi multimediali vengono divisi in due categorie:

- **Streaming:** la fruizione dei contenuti può iniziare prima che l'intero insieme di dati (file) sia stato ricevuto.
- **Bulk transfer** (trasferimento in blocco): i contenuti possono essere utilizzati solo dopo che l'intero file è stato ricevuto.

Nei servizi streaming è importante controllare **latenza** (ritardo tra il momento in cui i dati sono disponibili alla sorgente e quello in cui sono disponibili alla destinazione) e **jitter** (variabilità della latenza). Nei servizi bulk transfer conta soprattutto la qualità.

Servizi streaming e bulk transfer possono a loro volta essere suddivisi in:

- **Stored:** il contenuto è prodotto in un primo momento, e trasmesso asincronamente agli utenti.
- **Live:** il contenuto è prodotto immediatamente prima della trasmissione.

Esiste anche la categoria **real-time**, si tratta di un caso speciale di live in cui il ritardo è $< 150ms$. A questa categoria appartengono solo servizi streaming, non è possibile realizzare servizi multimediali bulk transfer real-time.

	Stored	Live	Real-time
Streaming	Visione di film, episodi, trasmissioni registrate	Internet radio Internet TV	Chiamate audio, Video chiamate, Video conferenza, DAD
Bulk Transfer	Podcast P2P file sharing FTP HTML	Messaggi testuali E-mail Messaggi audio Messaggio con immagini Messaggio con video	

Figura 1.1: Esempi di servizi multimediali divisi per categorie.

1.2 La creazione dei segnali multimediali: ADC

Molte sorgenti multimediali sono di natura segnali analogici, ovvero sono funzioni di variabile continua ed ampiezza continua. Per poter utilizzare questi segnali nei sistemi digitali è necessario **numeralizzare** o **digitalizzare** tali segnali. Di ciò si occupa la **conversione analogico-digitale (Analog Digital Conversion)**. La conversione analogico digitale consiste in due operazioni:

1. **Campionamento:** la variabile indipendente diventa discreta.
2. **Quantizzazione:** l'ampiezza diventa discreta.

Successivamente il segnale deve essere rappresentato sotto forma di una stringa di bit. Questa operazione viene fatta da una terza operazione:

3. **Codifica**: l'ampiezza del segnale viene rappresentata usando dei bit.

Il convertitore analogico digitale deve essere realizzato in modo tale che la perdita di informazione sia trascurabile. In questo modo è possibile ricavare il segnale analogico di partenza dal segnale digitalizzato. Non è importante che il segnale analogico risulti esattamente identico a quello di partenza, ci basta che questo sia percepito come identico.

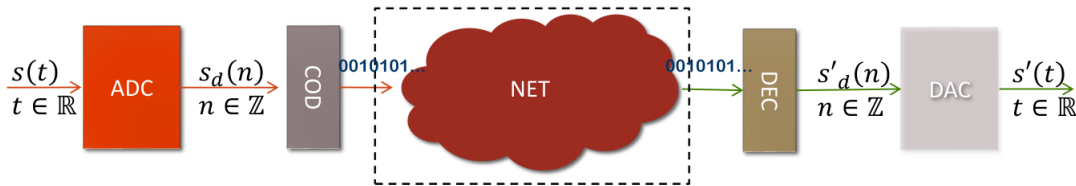
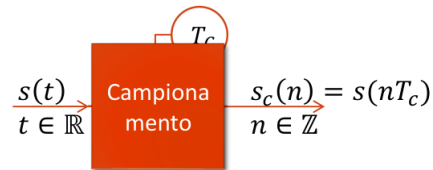


Figura 1.2: La catena di trasmissione dell'informazione.

1.2.1 Campionamento

Il campionamento consiste nel prelevare ad intervalli regolari dei valori di un segnale di variabile continua. L'intervallo di tempo viene chiamato periodo di campionamento T_c . Il risultato che si ottiene è una successione di valori.

La domanda che può sorgere spontanea è: il campionamento comporta una perdita di informazione? Il **teorema del campionamento di Shannon** risponde a questa domanda, ma per enunciarlo dobbiamo introdurre la **Trasformata di Fourier**.



La Trasformata di Fourier Dato un segnale a tempo continuo $s(t)$, sotto ipotesi molto larghe, è possibile calcolare lo **spettro** $S(f)$ cioè la sua trasformata di Fourier. Lo spettro $S(f)$ rappresenta il contenuto frequenziale di $s(t)$.

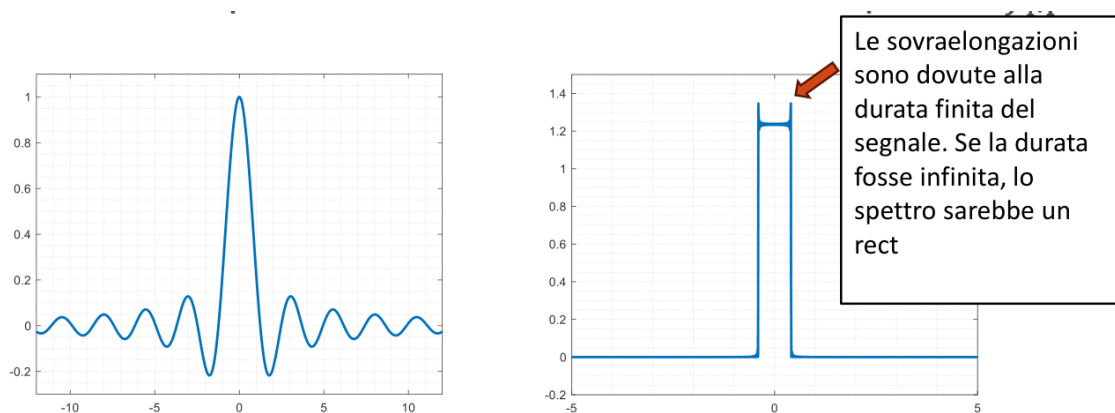


Figura 1.3: $s(t) = \text{sinc}(t) = \frac{\sin \pi t}{\pi t}$. Lo spettro $S(f)$ ha **supporto finito**. Si parla di **segnale a banda limitata**: non ha contributi frequenziali oltre una certa frequenza f_M .

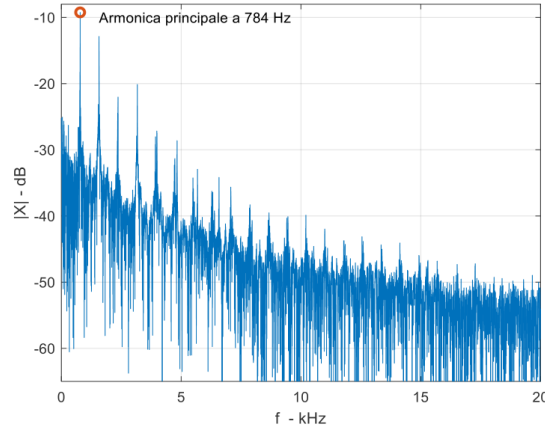


Figura 1.4: Spettro del segnale $x(t)$, nota SOL5 di pianoforte. Ascisse: frequenza in kHz . Ordinate: modulo della trasformata di Fourier in dB : $10 \log_{10} |X(f)|$. A $f_0 = 768 \text{ Hz}$ si trova un tono; le altre armoniche sono toni a frequenze multiple di f_0 .

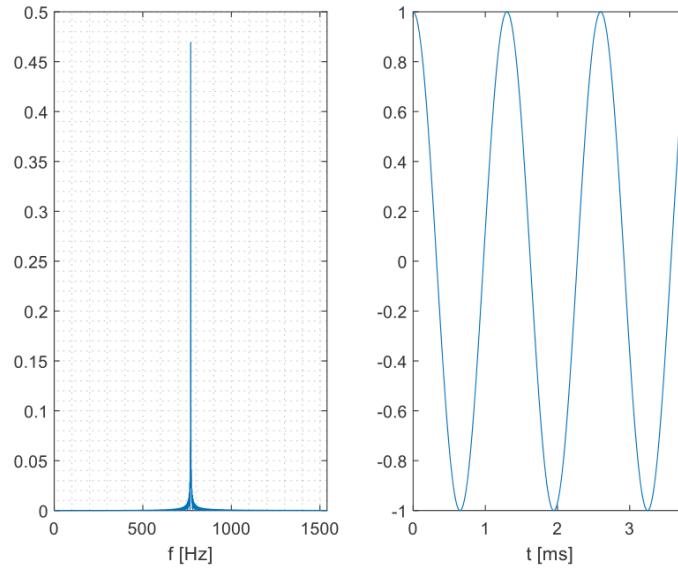


Figura 1.5: Tono puro: $s(t) = A \cos 2\pi f_0 t$. Il suo spettro è concentrato nella frequenza f_0 (impulso). Nel caso reale con segnali di durata T lo spettro assomiglia ad un impulso.

Teorema del campionamento di Shannon

Teorema 1.1 (Teorema del campionamento di Shannon). *Dato un segnale $s(t)$ a banda limitata (es: tono puro, nota SOL5 o $\text{sinc}(t)$) con frequenza massima f_M , esso può essere ricostruito con errore nullo a partire dalla successione numerica di campioni*

$$s_c(n) = s(t)|_{t=nT_c}$$

se e soltanto se la frequenza di campionamento $F_c = 1/T_c$ è almeno il doppio della frequenza massima f_M .

Dunque, i segnali a banda limitata costituiscono l'insieme di tutti e soli i segnali che possono essere perfettamente ricostruiti a partire dai loro campioni.

La relazione

$$F_c \geq 2f_M \quad (1.1)$$

è detta **criterio di Nyquist**. Se il criterio è soddisfatto, allora il teorema di Shannon ci fornisce anche la **formula d'interpolazione ideale**

$$s(t) = \sum_{n \in \mathbb{Z}} s(nT_c) \operatorname{sinc}\left(\frac{t - nT_c}{T_c}\right) \quad (1.2)$$

Usando tale formula possiamo ricostruire perfettamente un segnale a frequenza limitata a partire dai suoi campioni.

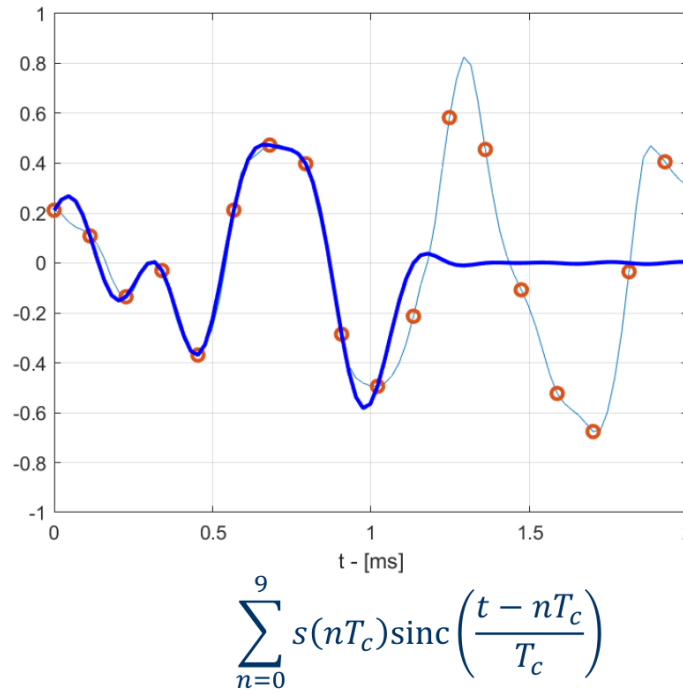


Figura 1.6: Esempio di ricostruzione di un segnale partendo dal campionamento.

In realtà, i segnali non sono mai rigorosamente a banda limitata. Infatti tutti i segnali a durata finita sono a banda rigorosamente illimitata. Tuttavia si può sfruttare il fatto che la maggior parte dell'informazione si concentra in una banda limitata.

Nei reali sistemi di conversione analogico digitale, prima del campionamento si effettua un **filtraggio passa-basso** che forza il criterio di Nyquist eliminando le frequenze al di sopra di $F_c/2$. Se non si effettua il filtraggio, e il segnale campionato non rispetta il criterio di Nyquist, allora il segnale ricostruito soffrirà di **aliasing**, cioè spostamento delle frequenze e quindi non si riesce a ricostruire il segnale originario.

1.2.2 Quantizzazione

Dopo che il segnale è stato campionato, il segnale $s_c(n)$ viene quantizzato: i valori del segnale vengono mappati su di un insieme discreto detto **dizionario** o **insieme dei livelli di quantizzazione**. Il numero di livelli di quantizzazione è indicato con L . Più L è grande e più aumenta la precisione di rappresentazione del segnale ma allo stesso tempo incrementa anche il **costo di codifica** R , ovvero il numero di bit necessari per rappresentare ogni campione.

$$R = \log_2 L$$

Quantizzazione uniforme Il tipo più comune di quantizzazione è la **quantizzazione** uniforme: un valore viene rappresentato con il più vicino multiplo di Δ .

$$Q(x) = \Delta \cdot \text{round}\left(\frac{x}{\Delta}\right)$$

Poniamo $x \in (-A, A)$, tale intervallo viene suddiviso in $\frac{2A}{\Delta}$ sotto-intervalli uguali, ognuno dei quali corrisponde ad un livello. Si ottengono dunque

$$L = \frac{2A}{\Delta}$$

livelli. Con la quantizzazione uniforme, l'**errore assoluto** di quantizzazione è al massimo $\frac{\Delta}{2}$.

L'operazione di quantizzazione è **irreversibile**¹. A differenza dal campionamento, si tratta di un'operazione che comporta ad un'irrimediabile perdita d'informazione. Per questo motivo i convertitori analogico digitale vengono progettati in modo che la quantizzazione sia molto precisa, cioè viene posto un valore Δ molto piccolo o equivalentemente un L molto grande.

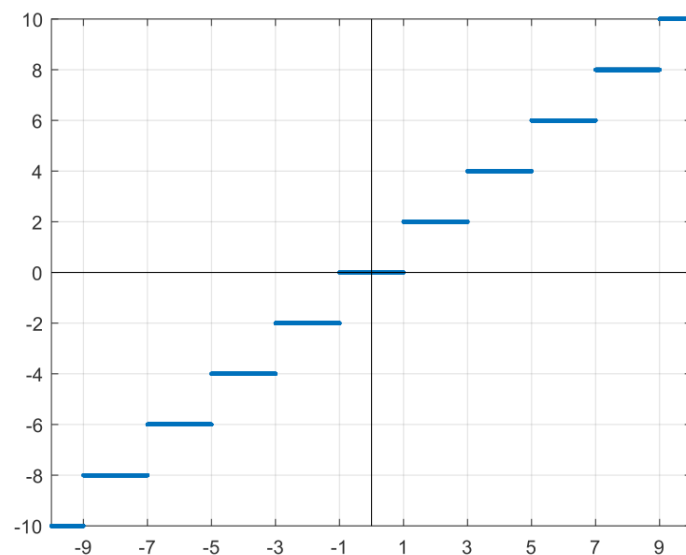


Figura 1.7: Relazione I/O di un quantizzatore uniforme con $\Delta = 2$.

Il quantizzatore deve essere progettato in maniera che questa perdita di informazioni non sia percepibile. I parametri Δ e L dipendono dal segnale che stiamo quantizzando, per esempio nell'audio 2^{16} livelli sono sufficienti per avere una percezione perfetta del suono, oppure per le immagini in bianco e nero bastano 2^8 livelli.

¹Non è irreversibile se quantizzo un segnale che è già stato quantizzato. Tuttavia non sono segnali reali che si trovano in natura. Per questi rimane vera l'irreversibilità.

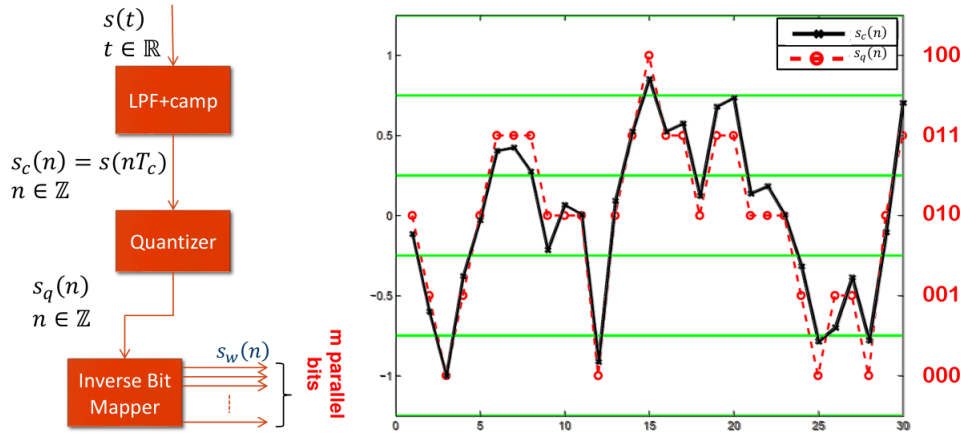


Figura 1.8: $s_c(n)$ quantizzato con $L = 2^3$, $A = 1$, $\Delta = 0.25$. Il modulo **inverse bit-mapping (IBM)** trasforma il valore discreto $s_q(n)$ nella corrispondente stringa di bit $s_w(n)$ (**binarizzazione**).

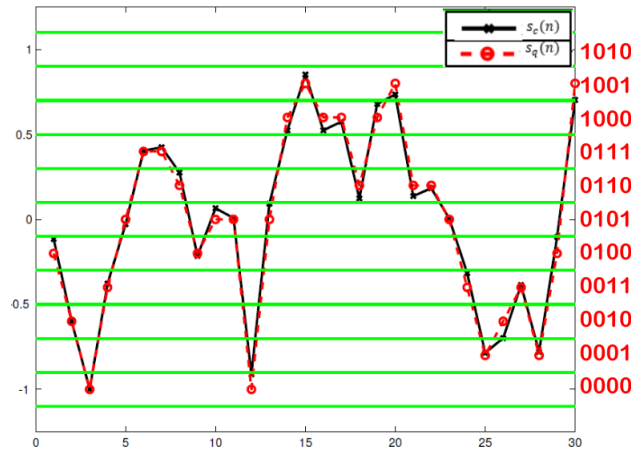


Figura 1.9: $s_c(n)$ quantizzato con $L = 2^4$, $A = 1$, $\Delta = 0.125$.

La (ri)quantizzazione Nei sistemi di trasmissione multimediale sono tipicamente presenti due quantizzazioni:

1. la prima avviene internamente all'ADC ed ha una distorsione molto piccola;
2. la seconda avviene internamente al **codificatore di sorgente**, dipende dall'algoritmo di compressione. Questa seconda quantizzazione viene chiamata riquantizzazione perché agisce su dati già quantizzati. Può introdurre una distorsione evidente.

1.3 Qualità di un segnale multimediale

Nella quantizzazione si fissano i parametri che permettono di definire il costo (numero di bit usati per quantizzare) e le prestazioni (qualità del segnale digitalizzato). Il costo si misura considerando il numero di bit per campione ($R = \log_2 L$), mentre misurare la qualità è più complesso.

Per misurare la qualità di un segnale digitalizzato, consideriamo la distorsione tra il segnale quantizzato s_q e il segnale (originale) campionato s_c . Esistono diversi modi per calcolare la

distorsione tra due segnali, un modo è quello di sfruttare l'**errore quadratico medio**

$$\text{MSE}(f, g) = \frac{1}{N} \sum_{n=0}^{N-1} [e(n)^2] = \frac{1}{N} \sum_{n=0}^{N-1} [f(n) - g(n)]^2 \quad (1.3)$$

dove: $n \in \{0, \dots, N-1\}$; $e(n) = f(n) - g(n)$ è il **segnale di errore** dato dalla differenza di due segnali (nel nostro caso $s_c(n)$, $s_q(n)$). Considerando f, g due vettori nello spazio euclideo, possiamo riscrivere l'MSE in funzione della distanza euclidea al quadrato dei due vettori

$$\text{MSE} = \frac{1}{N} \|e\|^2 = \frac{1}{N} \|f - g\|^2. \quad (1.4)$$

Sfruttando l'errore quadratico medio possiamo infine calcolare la qualità di un segnale calcolando il **PSNR (Peak Signal-to-Noise Ratio)**²

$$\text{PSNR} = 10 \log_{10} \frac{(2^b - 1)^2}{\text{MSE}} \quad (1.5)$$

dove il numeratore esprime la **potenza di picco** del segnale (massimo valore al quadrato): per segnali a valori positivi espressi su b bits, il valore è dato da $2^b - 1$.

1.4 Bit rate del segnale multimediale

In uscita dall'*Inverse Bit Mapper* viene collegato un convertitore parallelo/serie che serializza i bit in uscita: il segnale risultante è binario a tempo discreto. Se abbiamo un campione ogni T_c secondi, ogni campione viene rappresentato con $m = \log_2 L$ bits, allora in uscita dal convertitore ADC si ha un bit ogni

$$T_b = \frac{T_c}{m} = \frac{1}{mF_c} \leq \frac{1}{2mf_M}. \quad (1.6)$$

Il **bit-rate** è quindi

$$R = \frac{1}{T_b} = mF_c = F_c \log_2 L \quad (1.7)$$

1.5 La ricezione del segnale: DAC

Dopo che il segnale è stato digitalizzato, i bit prodotti vengono codificati o compressi e trasportati su rete. Compressione e trasporto potrebbero introdurre delle perdite d'informazione e per questo il segnale ricevuto $s'_d(n)$ può essere diverso da quello trasmesso. Per adesso trascuriamo queste problematiche e ci concentriamo sulla ricostruzione del segnale analogico partendo dal segnale ricevuto $s'_d(n)$, in altre parole studiamo la **conversione digitale-analogica**.

1.5.1 L'interpolazione

Prima dell'interpolazione il segnale digitale $s'_d(n)$ subisce due trasformazioni: viene parallelizzato e diventa $s'_w(n)$; l'**inverse Bit Mapping** trasforma la sequenza di bit ricevuti in ingresso e ritorna i valori quantizzati; il segnale diventa $s'_q(n)$.

L'**interpolazione** lavora a partire da $s'_q(n)$ e il suo scopo è quello di produrre i valori "intermedi" tra un campionamento e l'altro. Assumiamo l'errore di quantizzazione sia trascurabile

²Sarebbe il rapporto segnale rumore (SNR) calcolato considerando il picco del segnale invece che l'energia media.

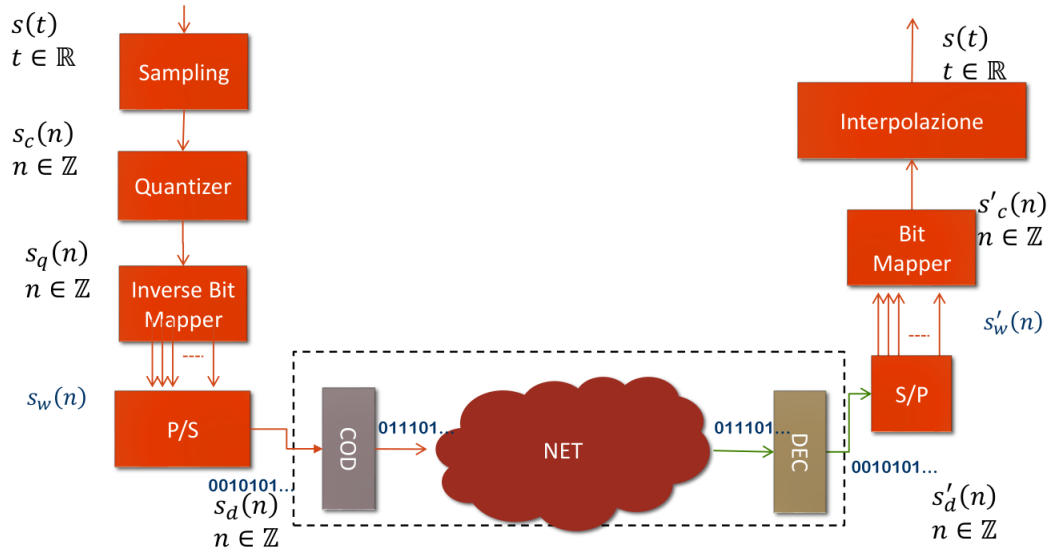


Figura 1.10: Trasmissione e ricezione di un segnale.

e quindi $s_q(n) = s(nT_c)$. Come abbiamo visto, la 1.2 è la formula d'interpolazione ideale fornita dal teorema di Shannon, tuttavia in pratica non è fisicamente realizzabile una funzione a supporto infinito. Per l'interpolazione si usano delle funzioni h , dette **nuclei** o **kernel** d'interpolazione.

$$\tilde{s}(t) = \sum_{n \in \mathbb{Z}} s(nT_c) h\left(\frac{t - nT_c}{T_c}\right). \quad (1.8)$$

Perché h sia un nucleo, deve rispettare due vincoli:

1. deve avere supporto finito;
2. deve valere $h(t) = \begin{cases} 1 & \text{se } n = 0 \\ 0 & \text{se } n \in \mathbb{Z}_0 \end{cases}$, in questo modo si ha che

$$\tilde{s}(kT_c) = \sum_{n \in \mathbb{Z}} s(nT_c) h\left(\frac{kT_c - nT_c}{T_c}\right) = \sum_{n \in \mathbb{Z}} s(nT_c) h(k - n) = s(kT_c).$$

Grazie a questo vincolo, l'interpolazione effettivamente passa negli istanti di campionamento e genera quelli intermedi.

Alcuni esempi di interpolazioni utilizzate in pratica sono: l'**interpolazione di ordine zero (Sample and Hold)**; l'**interpolazione di ordine uno (lineare)**; l'**interpolazione di ordine tre (cubica)**; l'**interpolazione con sinc troncato (cubica)**.

Quale nucleo di interpolazione conviene scegliere? Non c'è una risposta generale, dipende dall'applicazione che dobbiamo realizzare. In generale i parametri di progetto da considerare nella fase di ricezione sono: la frequenza di campionamento f_c e la scelta del nucleo d'interpolazione h .

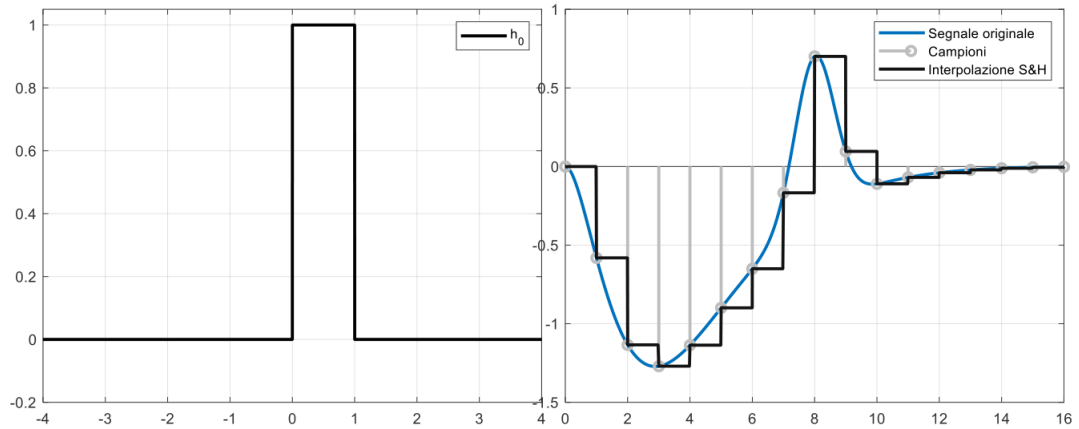


Figura 1.11: Interpolazione Sample and Hold. $h(t) = \text{rect}(t - 1/2)$. Implementazione hardware molto semplice.

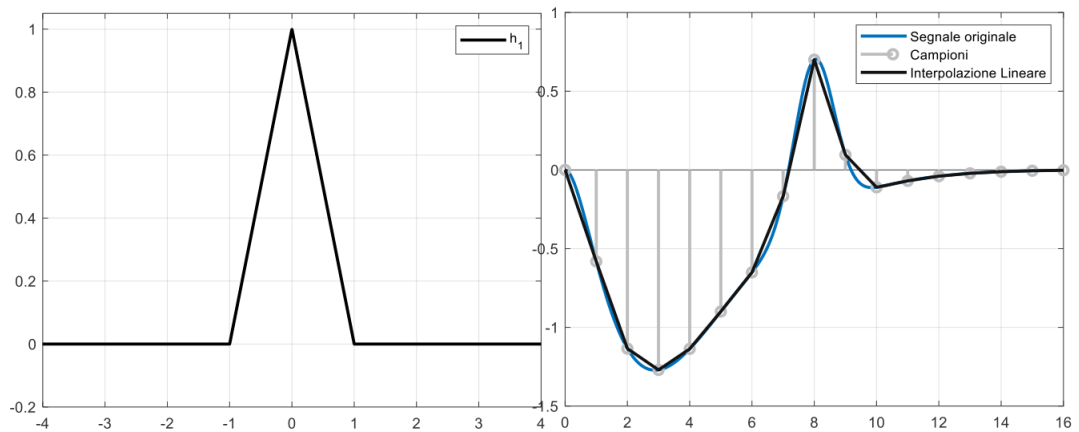


Figura 1.12: Interpolazione lineare. $h(t) = \text{triang}(t - 1)$. Più complessa del S&H ma offre una ricostruzione migliore.

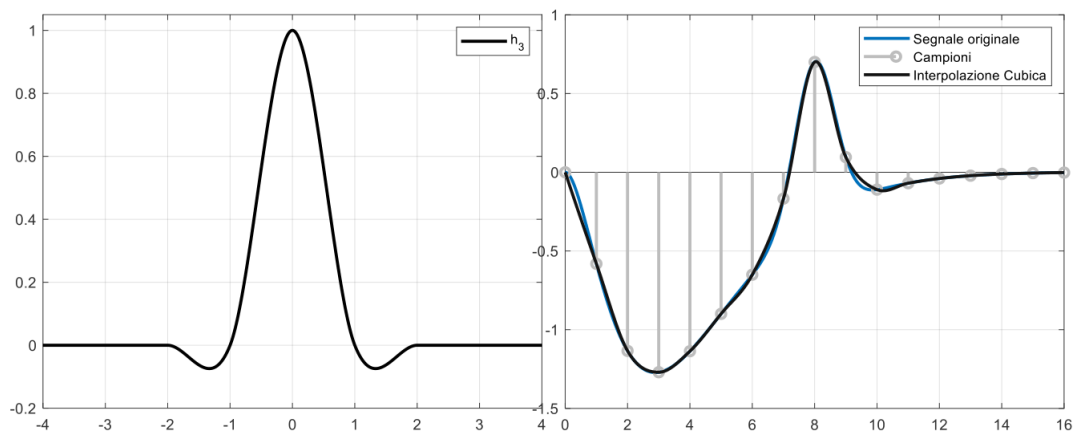


Figura 1.13: Interpolazione cubica. Molto usata per l'interpolazione digitale.

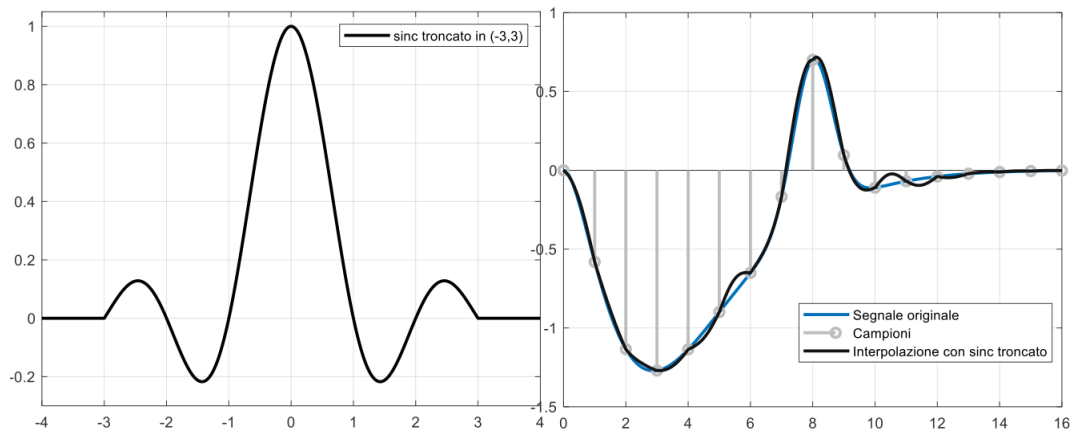


Figura 1.14: Interpolazione sinc troncato. $h(t) = \text{sinc}(t)\text{rect}(t/T)$. Spesso usato nelle schede audio, è il metodo più costoso ma più efficace.

Capitolo 2

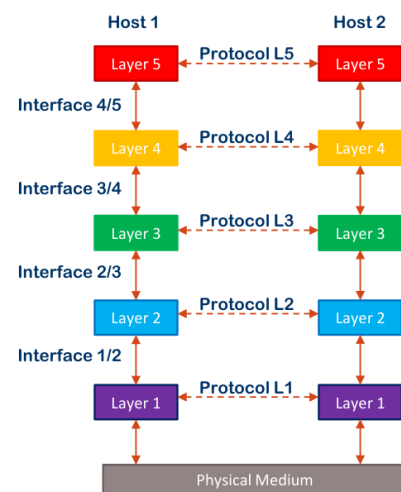
Servizi di rete per le applicazioni multimediali

L'obiettivo è quello di definire i concetti base che permettono di stabilire i parametri di progetto per una rete che supporti i servizi multimediali.

2.1 Architettura di rete a strati

Le reti sono organizzate come pile di livelli o strati. L'obiettivo di ogni livello è quello di fornire un insieme di servizi ai livelli superiori, nascondendo la complessità dell'implementazione. La definizione di quali servizi devono essere offerti all'interno di un livello è arbitraria, infatti esistono più modelli di architettura a strati (ISO/OSI, TCP/IP). Vediamo di cosa si occupano i vari livelli:

- servizi di **livello 5** (APP): applicazioni (di cui fanno parte i servizi multimediali);
- servizio di **livello 4** (Trasporto): canale a pacchetti end-to-end per i processi;
- servizio di **livello 3** (NET): canale a pacchetti end-to-end e senza connessione per gli host;
- servizio di **livello 2** (DLL): canale a pacchetti single-hop affidabile;
- servizio di **livello 1** (Phy): canale digitale single-hop inaffidabile.



L'interfaccia tra le applicazioni multimediali e la rete avviene a livello trasporto, dunque la realizzazione di un'applicazione richiede l'utilizzo delle primitive messe a disposizione dal livello trasporto.

Il **livello trasporto** è molto simile al livello di rete: entrambi forniscono un canale end-to-end ed entrambi possono fornire servizi orientati alla connessione o connectionless. Definiamo:

- *Servizi orientati alla connessione*: prevedono una fase di set-up della connessione, di trasmissione e poi di rilascio.

- **Servizi connectionless:** non necessitano di set-up e rilascio, i dati vengono frammentati in pacchetti trasferiti indipendentemente gli uni dagli altri.

Nell'architettura di Internet esiste un solo servizio di rete connectionless. Tra i compiti importanti che svolge il servizio di trasporto troviamo: gestione delle perdite di dati; gestione arrivo disordinato; congestione della rete.

2.2 Il livello trasporto e le applicazioni multimediali

Le applicazioni multimediali sfruttano un preciso servizio del livello di trasporto: un canale per flussi di bit, end-to-end e per processi. In particolare:

- **End-to-end:** l'applicazione multimediale non può occuparsi di definire il percorso seguito dal messaggio trasmesso, definisce soltanto chi sono il trasmettitore e il ricevitore.
- **Per processi:** significa identificare un processo di un host come sorgente e un processo su un host come destinatario.

Il servizio trasporto divide il flusso di bit prodotti dalla sorgente in unità di dati dette **pacchetti**. L'applicazione multimediale non dovrebbe esser a conoscenza della pacchettizzazione (secondo il principio dell'architettura stratificata), tuttavia in alcuni può dare delle indicazioni (es: bit per correggere gli errori) su come meglio suddividere i bit di stream in pacchetti. In questo modo viene facilitata la ricostruzione del messaggio in ricezione.

2.2.1 Requisiti di un'applicazione multimediale

In generale esistono 4 protocolli di livello 4. La scelta del protocollo da utilizzare dipende dai requisiti che deve soddisfare l'applicazione multimediale. I parametri da considerare per un'applicazione multimediale sono:

- **bit-rate:** bit al secondo trasmessi;
- **ritardo medio:** tempo che intercorre tra la trasmissione e la completa ricezione di un pacchetto;
- **varianza ritardo;**
- **probabilità errore sul bit;**
- **probabilità perdita di un pacchetto;**
- **garanzia (o assenza di garanzia) dell'ordine di ricezione dei pacchetti.**

A seconda dell'applicazione multimediale considerata, alcuni parametri potrebbero essere più importanti di altri.

Capitolo 3

La digitalizzazione di audio, video e immagini

L'obiettivo è quello di studiare il modulo COD di Figura 1.10. Lo scopo del COD è quello di comprimere il segnale $S_d(n)$ (ridurre il bit rate) andando a influenzare il meno possibile sulla qualità. In particolare ci interessa la qualità percepita: per capirla meglio dobbiamo studiare come l'utente finale percepisce i segnali multimediali.

3.1 La percezione del suono

Il nostro sistema uditivo effettua un'analisi in frequenza dei suoni, questo meccanismo può essere modellato come un *banco di filtri* in cui ogni filtro analizza una certa banda di frequenze. L'ascolto di un tono puro $x(t)$ a frequenza f_0 eccita la risposta $y_k(t)$ della k-esima fibra nervosa.

Esiste un valore di potenza minimo per il quale un tono puro a frequenza f viene percepito da una persona (varia da persona a persona) e si è notato che la nostra sensibilità è massima nella banda di frequenze della voce.

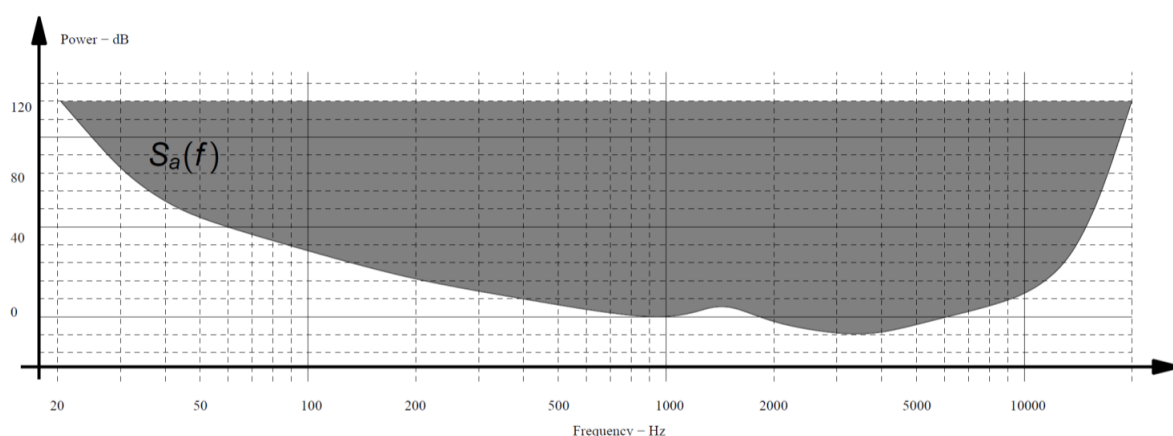


Figura 3.1: Soglia di potenza minima in funzione della frequenza di un tono.

L'immagine di Figura 3.1 mostra la soglia di potenza minima per percepire un segnale con una certa frequenza f . Tuttavia la soglia mostrata è valida solo se il tono è l'unico tono: in presenza di due toni si nota che quello di potenza maggiore può rendere impercettibile quello di potenza minore. Si parla di **mascheramento in frequenza** (Figura 3.2). In pratica quello che si nota è un incremento della soglia di percezione.

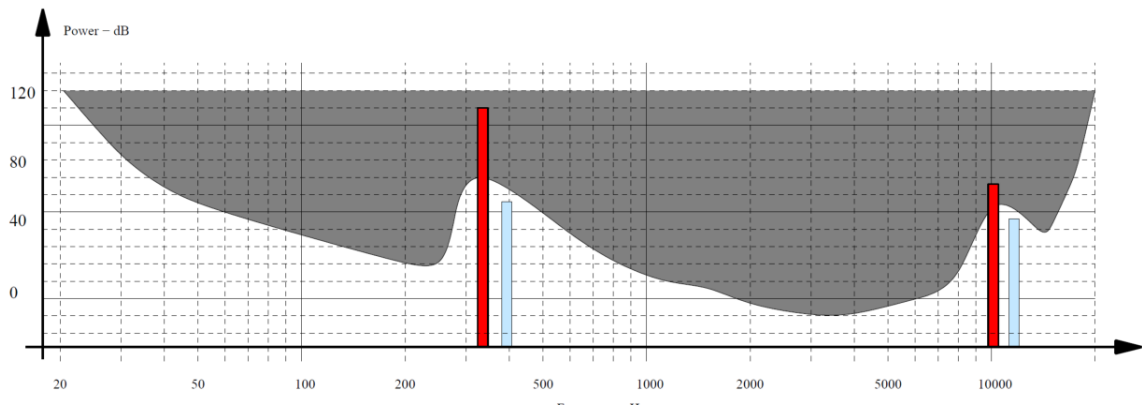


Figura 3.2: Soglia di potenza minima in presenza di due toni: il tono rosso maschera il tono azzurro.

Si nota inoltre che un tono tende a mascherare maggiormente le frequenze maggiori rispetto a quelle inferiori ().

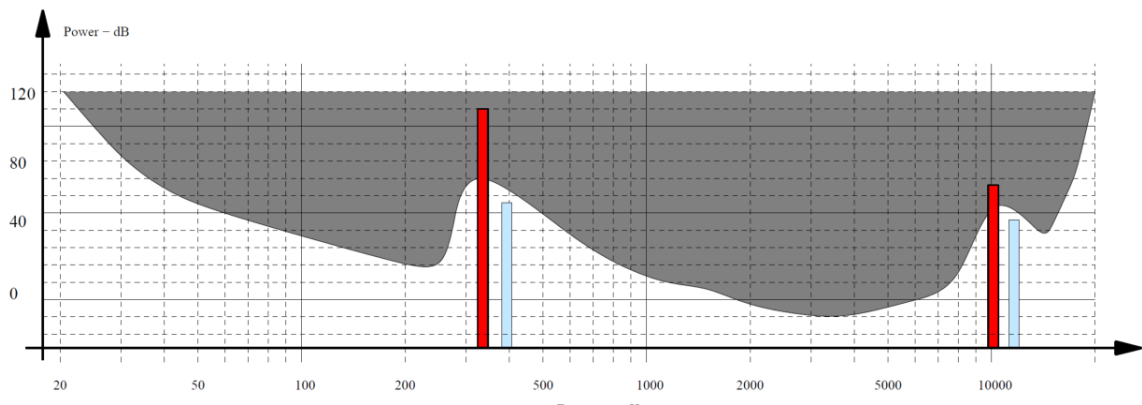


Figura 3.3: Pre-mascheramento e post-mascheramento di un tono. Pre-mascheramento: $2 \div 4ms$. Post-mascheramento: $100 \div 200ms$.

3.1.1 Codifica percettuale

L'idea della codifica percettuale è quella di comprimere (seconda quantizzazione) un segnale acustico in modo tale che il rumore generato venga mascherato da altre frequenze. Esistono diverse tecniche che mettono in pratica questo comportamento.

3.2 La digitalizzazione dei segnali audio

Il parlato (**segnale vocale**) viene trattato in maniera diversa rispetto ad altri tipi di segnali audio come per esempio la musica. Consideriamo i due casi separatamente.

3.2.1 Digitalizzazione tramite Pulse Code Modulation (PCM): segnali vocali

Un segnale vocale si concentra maggiormente nella banda $[200, 3400]Hz$. Solitamente vengono filtrati tramite un filtro passa-basso con frequenza di taglio $f_{cut} = 4kHz$ e frequenza

di campionamento $f_c = 2 \cdot 4kHz = 8kHz$ in modo da rispettare il criterio di Nyquist. Il numero di livelli di quantizzazione è stato determinato sperimentalmente, le scelte più comuni sono: $L = 128 = 2^7$, ovvero 7 bit per campione; oppure $L = 256 = 2^8$, ovvero 8 bit per campione. Infine il data rate che si ottiene è $R = f_c \log_2 L = 56kbps$ o $64kbps$. Per quanto riguarda la pacchettizzazione, un pacchetto corrisponde a $20ms$, quindi contiene $\frac{56kbps \cdot 20ms}{8b} = \frac{64kbps \cdot 20ms}{8b} = 160$ campioni per un totale di $160 \cdot 7b/8b = 140B$ o $160 \cdot 8/8B = 160B$.

Nel seguito considereremo solamente il caso con $L = 256$ livelli, quindi $R = 64kbps$ e dimensione del pacchetto $160B$. Questo schema viene chiamato **PCM** anche se a rigore non è una modulazione¹ ma uno schema di digitalizzazione.

3.2.2 Digitalizzazione della musica

Consideriamo **segnali musicali** tutti i segnali che non sono vocali, ovvero tutti i segnali che non hanno una banda inferiore a $4kHz$.

I segnali musicali si concentrano nella banda $[15, 20000]Hz$, i parametri del sistema di conversione analogico digitale che rendono impercettibile la distorsione sono: due canali (suono stereo); campionamento a $f_c = 44.1kHz$ con filtraggio anti-aliasing; quantizzazione su 16 bit per campione ($L = 2^{16}$). Vengono inoltre aggiunti dei bit di ridondanza per il riconoscimento e la correzione degli errori sui bit, in particolare si ha che il bitstream generato è composto dall'87% di dati e dal 13% di codice di protezione dagli errori.

Si trova dunque che il data rate è

$$R = f_c \log_2 L = 44.1kHz \cdot \log_2 2^{16} \cdot \underbrace{2}_{\text{canali}} = 1.411Mbps$$

mentre il bit-rate (R rappresenta l'87% del bit-rate)

$$R = R_T \cdot 0.87 \Rightarrow R_T = R/0.87 = 1.622Mbps$$

Segnale	Freq. camp.	Livelli	bit/campione	Canali	Data rate
Parlato	8kHz	256	8	1	64kbps
Musica	44.1kHz	65536	16	2	1.411Mbps

Tabella 3.1: Riassunto dei parametri per la digitalizzazione di segnali vocali e musicali.

Il **numero di canali** asseconda l'esigenza di *spazializzazione* del suono. La *stereofonia* (2 canali), al contrario delle *monofonia*, permette al nostro cervello di individuare la direzione di provenienza del suono.

3.3 Formati d'immagine e video non compressi

3.3.1 Campionamento spaziale, quantizzazione e HDR

Nell'acquisizione delle immagini entra in gioco il **campionamento spaziale**. In sostanza ogni pixel dell'immagine misura il colore della luce che colpisce la superficie del sensore ad esso associata, in questo modo si effettua il campionamento. Più pixel ha il sensore e più alta sarà la qualità dell'immagine.

Le immagini in bianco e nero possono essere rappresentate con un byte per pixel, in questo modo si ottiene una scala di 256 livelli di grigio (**quantizzazione**). L'immagine prodotta è

¹La modulazione associa ad ogni stringa di b bit una forma d'onda da trasmettere su un canale per una certa durata detta intervallo di simbolo.

indistinguibile da una versione della stessa immagine non quantizzata. In generale 256 livelli sono sufficienti se l'immagine non ha zone molto più luminose o molto meno luminose del resto, in questi casi non si ottiene una buona immagine se non si aumenta il numero dei bit per pixel. Questi formati vengono chiamati **High Dynamic Range (HDR²)** e possono usare 10 o 12 bit per campione.

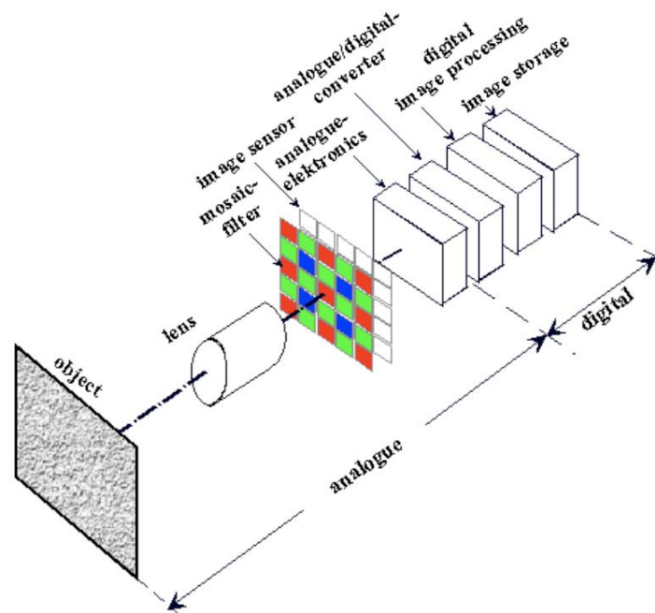


Figura 3.4: Processo di acquisizione delle immagini.

3.3.2 Immagini a colori

La percezione dei colori avviene grazie alla presenza di fotorecettori (chiamati *coni*) nella retina. Ci sono tre tipi di coni, ognuno dei quali ha una diversa risposta in funzione della **lunghezza d'onda** della luce che lo colpisce. Gli stimoli emessi da queste tre tipologie di coni formano un *tristimolo* che permette al nostro cervello di produrre la sensazione del colore.

Il colore di un pixel può essere rappresentato con **3 livelli d'intensità**, ognuno corrispondente ad uno dei 3 colori primari. La tripletta di valori risultante viene utilizzata per modulare la luce emessa dai led degli schermi. In questo modo è possibile rappresentare una vasta gamma di colori utilizzando solamente tre differenti tipologie di led.

Ognuno dei 3 canali (cioè delle 3 componenti **RGB**) è rappresentato su b bits. In genere si ha $b = 8$, in questo modo si possono rappresentare $(2^8)^3 \approx 16 \cdot 10^6$ diversi colori. Esistono anche i formati **HDR** a colori che permettono di rappresentare un numero ancora più elevato di colori.

²Il formato HDR dei cellulari non aumenta il numero di bit per pixel ma simula il funzionamento: vengono scattate più foto in modo da ottenere diversi livelli di esposizione dell'immagine e l'immagine risultante tiene conto di tutti gli scatti. In questo modo si ottengono immagini più precise.

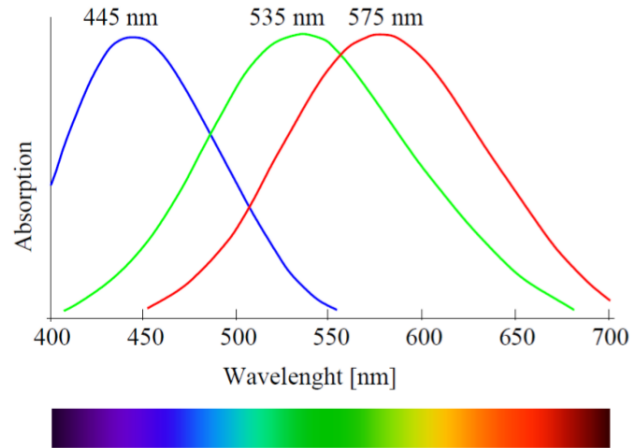


Figura 3.5: Assorbimento in funzione delle lunghezze d'onda della luce. Un colore a xnm può essere ottenuto con un'opportuna combinazione dei **colori primari** rosso, verde e blu.

3.3.3 Cambiamento di spazio: spazio RGB e YCbCr

Nello **spazio RGB** il colore di un pixel in posizione (n, m) è dato dalla seguente tripletta di valori:

$$x(n, m) = \begin{bmatrix} r(n, m) \\ g(n, m) \\ b(n, m) \end{bmatrix}$$

Applicando una rotazione degli assi si ottengono altri spazi di colore più utili per le applicazioni multimediali.

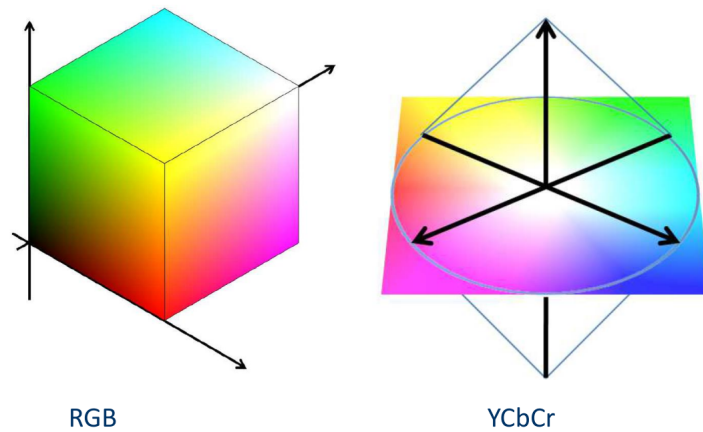


Figura 3.6: Spazio di colori RGB e YCbCr.

Lo **spazio YCbCr** si ottiene mediante una rotazione degli assi dello spazio RGB

$$\begin{bmatrix} y \\ c_b \\ c_r \end{bmatrix} = T \begin{bmatrix} r \\ g \\ b \end{bmatrix}.$$

In pratica la diagonale principale del cubo nello spazio RGB diventa uno degli assi nello spazio YCbCr. Nello spazio YCbCr individuiamo:

- **luminanza**: rappresentata dalla componente y , indica la luminosità del pixel (indipendentemente dal colore);
- **crominanza**: rappresentate dalle componenti c_b , c_r , da informazioni sul colore del pixel.

Lo spazio YCbCr, a differenza dello spazio RGB, permette di lavorare con segnali sparsi. Definiamo:

- **segna non sparso**: un segnale in cui tutti i campioni hanno pari importanza;
- **segnale sparso**: segnale in cui i campioni non hanno tutti la stessa importanza e quindi si può decidere (tramite quantizzazione) di scartarne qualcuno senza perdere qualità percepita. Non è possibile fare lo stesso ragionamento con i segnali non sparsi.

Siccome i nostri occhi sono molto meno sensibili alla crominanza rispetto alla luminanza, è possibile scartare i 3/4 dei campioni di crominanza senza produrre perdita di qualità percepita. Quindi il segnale è sparso dal punto di vista percettivo: i campioni importanti sono quelli della luminanza; i campioni meno importanti sono quelli della crominanza. Andando a quantizzare finemente i campioni importanti e più grossolanamente i campioni meno importanti si ottiene una migliore compressione rispetto a quella che si può ottenere con un segnale non sparso.

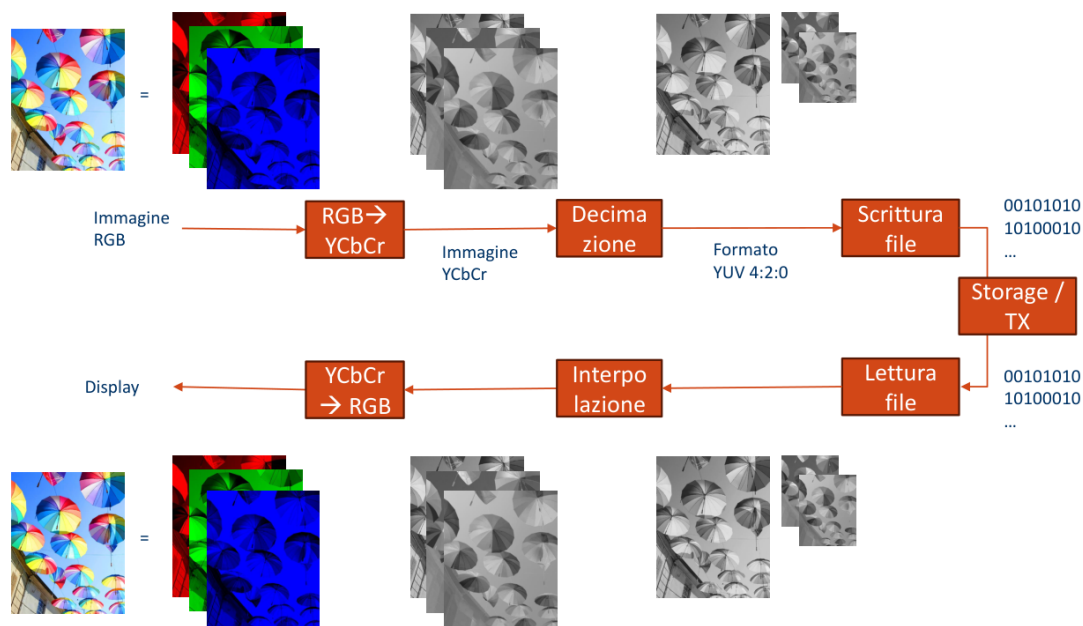


Figura 3.7: Pipeline.

Dalla Figura 3.7:

- tramite prodotto tra matrici passo dallo spazio RGB a quello YCbCr;
- scarto i 3/4 dei campioni di crominanza c_b e c_r e mantengo tutti i valori di luminanza y (**decimazione**). In questo modo riduco la dimensione dei dati da memorizzare;
- in fase di lettura vado a ricostruire i campioni di crominanza scartati nella fase di decimazione utilizzando un'interpolazione (è sufficiente l'interpolazione Sample And Hold)
- tramite prodotto tra matrici passo dallo spazio YCbCr allo spazio RGB e infine visualizzo l'immagine.

L'immagine mostrata sul display non è esattamente identica a quella di partenza a causa della decimazione, ma l'occhio umano non percepisce la differenza.

Se l'immagine di partenza era di 10Mpixel , allora in byte pesa

$$\text{size}_{\text{RGB}} = 10\text{Mpixel} \cdot 3B/\text{pixel} = 30\text{MB}$$

se utilizziamo il formato RGB, mentre utilizzando il formato YCbCr e scartando $3/4$ dei campioni di cromaticità, il peso è di

$$\text{size}_{\text{YCbCr}} = 10\text{Mpixel} \cdot 1B/\text{pixel} + 10\text{Mpixel}/4 \cdot 1B/\text{pixel} + 10\text{Mpixel}/4 \cdot 1B/\text{pixel} = 15\text{MB}$$

3.4 Formati video

I video sono flussi continui di immagini chiamate **frames**. Il valore minimo di 24 fps assicura un'impressione di fluidità del movimento.

Nome formato	Risoluzione		Numero pixel	Immagini al secondo	Tasso [Mbps]
CIF	352	288	101376	30	36
SDTV 576p	720	576	414720	60	299
HDTV 720p	1280	720	921600	50	553
HDTV 1080p	1920	1080	2073600	50	1244
2K	2048	1080	2211840	50	1327
4K	4096	2160	8847360	50	5308
8K	7680	4320	33177600	50	19907

Figura 3.8: Esempi di formati video non compressi.

Capitolo 4

Codifica del segnale vocale

Capitolo 5

Codifica lossless

Una codifica lossless permette di rappresentare i simboli emessi da una sorgente in modo perfettamente invertibile, cioè senza perdita di informazioni.

5.1 Nozioni di base

5.1.1 Sorgente, alfabeto, codice e codeword

Consideriamo con X una sorgente che emette dei simboli appartenenti ad un certo insieme finito detto **alfabeto** di X , utilizzeremo la notazione $A_X = \{x_1, \dots, x_M\}$. La sorgente viene modellata come un **processo aleatorio**, ovvero una sequenza di variabili aleatorie.

Un **codice** C è definito come un'applicazione tra l'alfabeto della sorgente e l'insieme delle stringhe di bit di lunghezza finita.

$$C : x_i \in A_X \rightarrow c_i \in \{0, 1\}^*$$

Ogni simbolo è caratterizzato dalla probabilità $P(X = x_i) = p_i$. Le stringhe c_i sono dette **codeword** e sono caratterizzate dalla lunghezza l_i .

5.1.2 Codici a lunghezza variabile e lunghezza media di un codice

Nei codici a lunghezza fissa tutte le codeword hanno la stessa lunghezza; per i **codici a lunghezza variabile** definiamo la **lunghezza media** come segue:

$$\mathcal{L} = \sum_{i=1}^M p_i l_i$$

Si preferiscono i codici a lunghezza media piccola (non è l'unico requisito).

5.1.3 Codici a prefisso

Risulta fondamentale utilizzare codici **univocamente decodificabili**, in modo tale che ogni sequenza di codeword possa essere decodificata in un solo modo.

I **codici a prefisso** sono codici in cui nessuna codeword è prefisso di un'altra, questi codici sono univocamente decodificabili. Si tratta di codici **istantanei**: quando si trova una codeword nel bitstream codificato, si può immediatamente decodificare il simbolo.

Nel seguito considereremo soltanto codici istantanei (cioè a prefisso) perché si può dimostrare che il miglior codice istantaneo ha la stessa $\mathcal{L}$ del miglior codice univocamente decodificabile.

5.1.4 Codice ottimo

Tra tutti i codici istantanei si vuole trovare il **codice ottimo**, ovvero il codice che ha la lunghezza media $\mathcal{L}$ minima. Il **teorema di Shannon** sulla codifica di sorgente ci permette di trovare la lunghezza minima, il **codice di Huffman** permette di ricavare il codice ottimo.

5.2 Cenni di teoria dell'informazione

5.2.1 Entropia di una variabile aleatoria

Sia data una variabile aleatoria X a valore discreti nell'insieme $A_X = \{x_1, \dots, x_M\}$ e sia $p_i = P(X = x_i)$. L'**informazione** associata all'evento $X = x_i$ è definita come

$$i(x_i) = \log_2 \frac{1}{p_i} = -\log_2 p_i.$$

L'informazione è una quantità non negativa e funzione decrescente della probabilità. Se due eventi sono indipendenti, allora la loro informazione è la somma delle due informazioni.

L'informazione media è detta **entropia**:

$$H(X) = E[i(X)] = \sum_{i=1}^M p_i \log_2 \frac{1}{p_i}.$$

L'entropia può essere vista come il grado di incertezza sulla realizzazione di una variabile aleatoria. In questo senso: la massima incertezza si ottiene quando i simboli sono equiprobabili (condizione di **non sparsità**); l'incertezza diminuisce se esistono pochi simboli ad alta probabilità e molti a bassa probabilità (condizione di **sparsità**).

L'entropia è indipendente dai valori dell'alfabeto della variabile aleatoria, è legata solamente alla probabilità che questi assumono. Per cambiare l'entropia associata ad una variabile aleatoria dobbiamo agire sull'alfabeto. Per esempio possiamo considerare coppie di valori (es: coppie di lettere dell'alfabeto) e non valori singoli (es: lettere singole): in questo modo la probabilità può cambiare e di conseguenza anche l'entropia.

5.2.2 Teorema di Shannon sulla codifica di sorgente

Il teorema di Shannon sulla codifica di sorgente pone un limite inferiore sulla lunghezza media di un codice istantaneo ottimo.

Teorema 5.1. *La lunghezza media $\mathcal{L}^*$ del codice istantaneo ottimo per una sorgente aleatoria X è vincolata da*

$$H(X) \leq \mathcal{L}^* < H(X) + 1$$

e l'uguaglianza si ottiene se e solo se tutte le probabilità dei simboli dell'alfabeto di X si possono scrivere come potenza intera negativa di due (distribuzione diadica).

Il teorema di Shannon non è costruttivo, non ci dice come generare il codice ottimo data una distribuzione di probabilità. Non ci dice nemmeno, in caso di distribuzione non diadica, il valore della lunghezza ottima $\mathcal{L}^*$. A queste due domande risponde la **codifica di Huffman**.

5.3 Codifica di Huffman

La codifica di Huffman permette di trovare uno tra i codici ottimi (non ne esiste uno unico ma diversi equivalenti) associati ad una data distribuzione di probabilità. Date le probabilità dell'alfabeto di sorgente, si costruisce un albero binario con un algoritmo iterativo: la

codeword di un simbolo si ottiene leggendo i bit associati ai rami del percorso che portano dalla radice alla foglia associata al simbolo. Il codice che si ottiene è a prefisso e se ne può dimostrare l'ottimalità.

Il codice di Huffman non sfrutta la dipendenza statistica che può esistere tra simboli consecutivi emessi da una sorgente. Per sfruttare tale dipendenza e comprimere ulteriormente i dati, si può ricorrere alla **codifica a blocchi**.

5.4 La codifica a blocchi

Risulta conveniente raggruppare i simboli in blocchi di dimensione K , indicati con X^K . In questo modo l'entropia tipicamente decresce. Dal teorema di Shannon si ricava che:

$$H(X^K) \leq \frac{\mathcal{L}}{K} = \mathcal{L}_S < \frac{H(X^K)}{K} + \frac{1}{K}.$$

Al crescere di K si ha:

$$\mathcal{H}(X) = \lim_{K \rightarrow \infty} \frac{H(X^K)}{K} \leq H(X)$$

tale limite viene chiamato **tasso entropico** ed esiste sotto opportune ipotesi: rappresenta il limite ultimo della codifica lossless. Prendendo un K abbastanza grande, la lunghezza media per simbolo tende al tasso entropico invece che all'entropia, e il tasso entropico è inferiore all'entropia o nel caso peggiore uguale ad essa.

5.5 Codifica aritmetica

Dato che la complessità della codifica di Huffman è esponenziale con K dimensione del blocco di simboli, si può usare una codifica subottima: la **codifica aritmetica**. Si dimostra che la lunghezza media per una codifica aritmetica è

$$\mathcal{L}_A < H(X) + 2.$$

Il vantaggio della codifica aritmetica è che la complessità cresce linearmente con K . La lunghezza media per simbolo è data da

$$\mathcal{L}_A < \frac{H(X)}{K} + \frac{2}{K} \rightarrow \mathcal{H}(X) \leq H(X).$$

Un vantaggio della codifica aritmetica è che è **adattiva**: la distribuzione può essere modificata man mano che si procede nella codifica (e decodifica), aumentando le probabilità dei simboli più frequenti. L'aggiornamento delle probabilità deve essere identico al codificatore ed al decodificatore.

La codifica aritmetica adattiva basata sul contesto rappresenta lo stato dell'arte della codifica lossless per i segnali multimediali.

5.6 Tipi di codifica lossless comunemente utilizzati

5.6.1 La codifica Exp-Golomb

La codifica Exp-Golomb si usa quando l'alfabeto d'ingresso è costituito dai numeri naturali o dai numeri interi. Non è un codice basato su di un modello esplicito di probabilità (non è quindi ottimo). Al contrario, può essere usato quando non si conosce la distribuzione esatta delle probabilità.

Exp-Golomb: numeri naturali (unsigned EGC) Per codificare un numero naturale n (unsigned EGC) si procede in questo modo:

- se $n = 0$, $c_U(n) = 1$;
- altrimenti, sia s la stringa di bit che rappresenta il numero $n + 1$ in binario usando il minimo umero di bit. Tale numero è $b = \lfloor \log_2(n + 1) \rfloor + 1$. Infine, $c_U(n)$ è data da $b - 1$ zeri (leading zeros) seguiti dalla stringa s .

n	$n + 1$ on b bits	Leading zeros	$c_U(n)$
0	1	-	1
1	10	0	010
2	11	0	011
3	100	00	00100
4	101	00	00101
5	110	00	00110
6	111	00	00111
7	1000	000	0001000
...	...	...	...

Exp-Golomb: numeri interi (signed EGC) Per codificare il numero intero n (signed EGC) si procede in questo modo: Per prima cosa si mappa l'intero con segno n nell'intero senza segno $m(n)$ secondo la regola seguente:

$$m(n) = \begin{cases} 2n - 1 & \text{se } n > 0 \\ |2n| & \text{se } n \leq 0 \end{cases}.$$

Dopodiché si usa la codifica unsigned EFC su $m(n)$, quindi $c_S(n) = c_U(m(n))$.

n	$m(n)$	$c_S(n) = c_U(m(n))$
0	0	1
1	1	010
-1	2	011
2	3	00100
-2	4	00101
3	5	00110
-3	6	00111
4	7	0001000

5.6.2 Codifica per categorie e ampiezza

Essa permette di ridurre il numero di bit per la codifica di numeri grandi, a spese del numero di bit per la codifica di numeri piccoli, che invece aumenta. Si tratta della codifica che viene utilizzata in JPEG.

Esempio di codifica per categoria e ampiezza Si definiscono 11 categorie, codificate con un opportuno codice a prefisso.

1. La categoria di n è $k = \lceil \log_2(|n| + 1) \rceil$. Nella prima categoria c'è solo il valore 0, nella seconda ci sono ± 1 nella terza $\pm 2, \pm 3$, nella quarta $\pm 4, \pm 5, \pm 6, \pm 7$, etc.
2. Si emette il codice della categoria.
3. Si codifica $|n|$ in binario su k bit e se $n < 0$ si prende il complemento della stringa ottenuta.

Poniamo di voler codificare il numero -6 . Si ha che: la categoria è $3 \rightarrow 100$; $|-6| = 6 \rightarrow 110$ ma siccome il segno è negativo si considera il complemento $\rightarrow 001$. Dunque si ottiene che la codifica di -6 è 100001.

Category	Code
0	00
1	010
2	011
3	100
4	101
5	110
6	1110
7	11110
8	111110
9	1111110
10	11111110
11	111111110

Category	Amplitude	Code
0	0	-
1	+1	1
	-1	0
2	+2	10
	+3	11
	-2	01
	-3	00
	...	...
3	+4	100
	+5	101
	...	...
	-7	000
...	...	...
11	1024	1 000 0000 000
	...	...
	-2047	0 000 0000 000

5.6.3 Codifica con dizionario

Utilizzate nei software "*zip" (es: bzip, gzip, winzip, etc.). L'idea è di creare una tabella con le sequenze di simboli più comuni.

- Il modo in cui la tabella è creata deve essere ripetibile dal decodificatore.
- Più si procede nella codifica e più le sequenze comuni diventano lunghe, permettendo una rilevante diminuzione della dimensione dei dati.

Efficace per file di testo, poco efficace per dati multimediali.

5.6.4 Codifica lossless predittiva

Caratteristiche: Se i dati presentano una dipendenza statistica è possibile creare la predizione del simbolo corrente a partire dai simboli già codificati (e quindi noti anche al decodificatore). Se la predizione è fatta in modo coerente con la natura dei dati, l'errore di predizione avrà una piccola entropia (dati sparsificati). Si può allora usare un qualsiasi codificatore entropico sull'errore di predizione invece che sui dati.

Capitolo 6

La compressione d'immagini e lo standard JPEG

6.1 Introduzione

Abbiamo visto che la codifica lossless permette di rappresentare i simboli emessi da una sorgente in modo perfettamente invertibile, cioè senza perdita d'informazione. Tuttavia, la codifica lossless applicata alle immagini, non ci permette di ottenere una significativa riduzione della dimensione. La soluzione è quella di utilizzare una **codifica con perdita d'informazione** che introduce un certo grado di degradazione del segnale rappresentato ma permette una migliore compressione.

Ricordiamo che i segnali che andiamo a codificare in questa fase sono segnali multimediali digitalizzati che hanno già subito un processo di conversione analogico digitale (progettato in modo tale da non generare distorsioni visibili o udibili).

Oltre al tasso (misura del costo) ed al PSNR (misura della qualità, vedi sezione 1.3), è importante considerare altri parametri di un sistema di compressione, tra i quali:

- **complessità computazionale:** requisiti di memoria e numero di operazioni;
- **ritardo:** se è necessario accumulare dati prima di poter cominciare la codifica;
- **robustezza:** indica quanto si degrada il segnale se ci sono errori nel bitstream codificato.

Tipicamente, migliorare uno dei parametri comporta un peggioramento in uno o più degli altri.

6.2 La trasformata coseno discreta

Per ridurre il tasso di codifica di un segnale, l'unico strumento a disposizione è la quantizzazione. Ma, siccome i segnali multimediali sono già stati quantizzati dal CAD, quantizzare ulteriormente causa una rapida degradazione del segnale. La soluzione è quella di trovare una **rappresentazione sparsa** del segnale e quantizzare quest'ultima: quantizzare i dati "non importanti" della rappresentazione sparsa porta ad una degradazione impercettibile del segnale. Abbiamo visto l'applicazione di questo principio nella decimazione della crominanza nello spazio YCbCr (sottosezione 3.3.3).

6.2.1 Trasformate lineari ortogonali

Un modo per rendere sparso un segnale è quello di **applicare una trasformata lineare**. L'idea è quella di:

- suddividere il segnale in blocchi di N campioni:

$$x = [s(n+1), \dots, s(n+N)]^T \in \mathbb{R}^N$$

- applicare una **trasformata lineare ortogonale** (rotazione degli assi):

$$y = Ax$$

- infine quantizzare separatamente le componenti della trasformata:

$$\tilde{y} = [Q(y(1), \Delta_1), \dots, Q(y(N), \Delta_N)].$$

Se la trasformata è *sparsificante*, allora molti valori di y saranno piccoli e quantizzarli (cioè approssimarli a zero) introduce un piccolo errore.

Utilizzando trasformate ortogonali, si ha che $A^{-1} = A^T$ e la ricostruzione del segnale si ottiene applicando la trasformata inversa A^{-1} al segnale quantizzato:

$$\tilde{x} = A^{-1}\tilde{y} = A^T\tilde{y}.$$

In questo modo si ottiene che l'errore risultante è

$$\text{MSE} = \frac{1}{N} \|y - \tilde{y}\|^2.$$

In altre parole, l'utilizzo di trasformate ortogonali permette di conservare le distanze, e quindi un piccolo errore su y comporta un piccolo errore su x . Dunque conviene applicare una strategia di quantizzazione ottimale direttamente sulla trasformata.

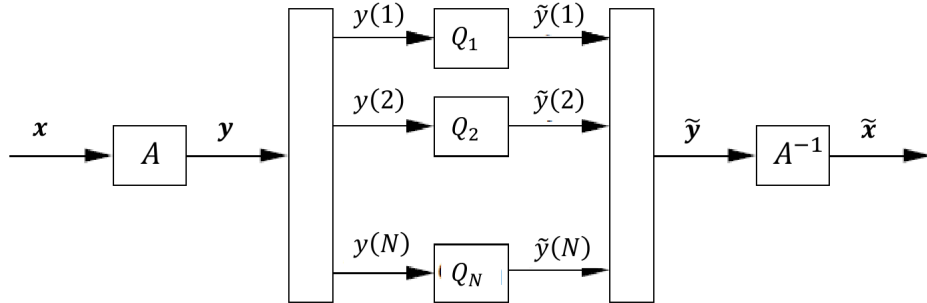


Figura 6.1: Schema della codifica con trasformata.

6.2.2 Scelta della trasformata

Come abbiamo detto, il requisito è che la trasformata sia ortogonale e sparsificante. Una possibile scelta di trasformata è la **trasformata di Fourier discreta (DFT)** applicata alle righe ed alle colonne di una immagine. In questo modo si ottengono le **frequenze spaziali**, ovvero le velocità di variazione in orizzontale e in verticale (Figura 6.3).

Una variante della DFT è la **trasformata coseno discreta (DCT)**. LA DCT produce valori reali, le frequenze sono solo positive e permette una migliore sparsificazione (Figura 6.4).

Al fine di effettuare delle analisi sulle frequenze locali, si può sfruttare la DCT sui blocchi dell'immagine invece che sui singoli pixel. Così facendo si nota che il segnale è maggiormente sparso nelle parti dell'immagine in cui non ci sono dettagli o bordi (Figura 6.5).

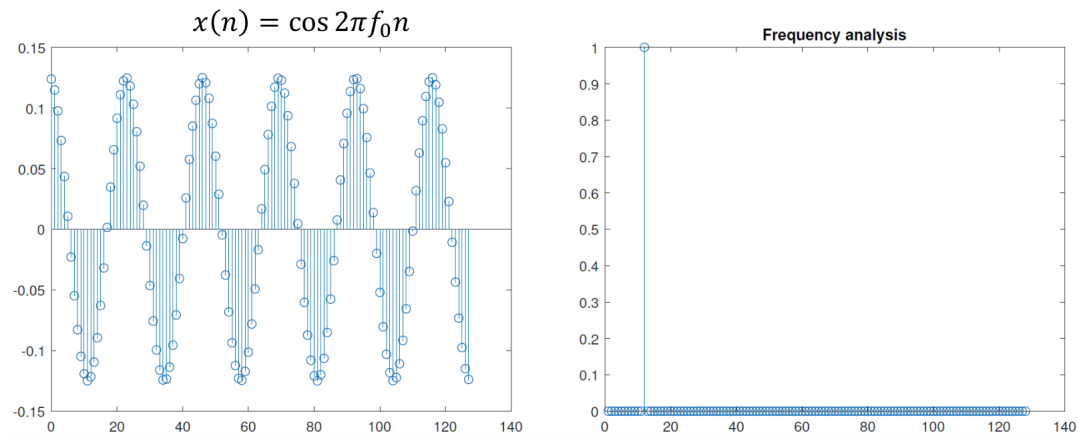


Figura 6.2: A sinistra un vettore x di N campioni di un segnale sonoro (tono puro). Il segnale è non-sparso: tutti i campioni hanno stessa importanza. A destra il vettore "trasformato" $y = Ax$ dove A è una matrice ortogonale che consente di eseguire un'analisi in frequenza. Il segnale risultante è sparso: un campione non è trascurabile, $N - 1$ lo sono.

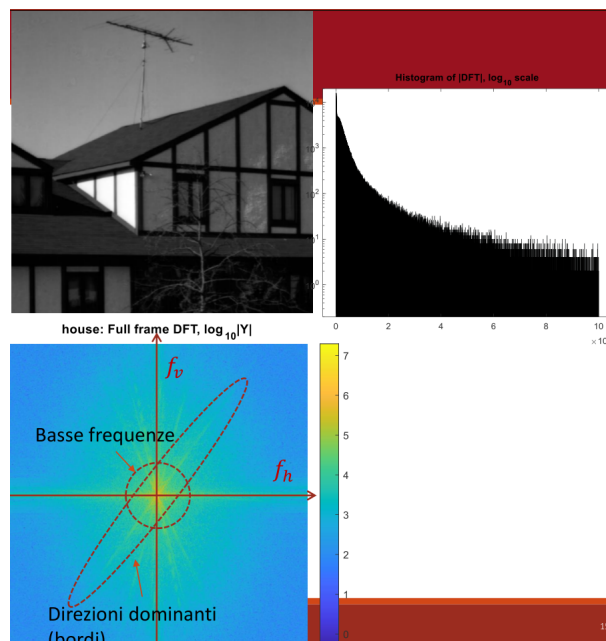


Figura 6.3: Trasformata di Fourier discreta (DFT) applicata alle righe ed alle colonne dell'immagine. L'istogramma mostra la sparsità della trasformata.

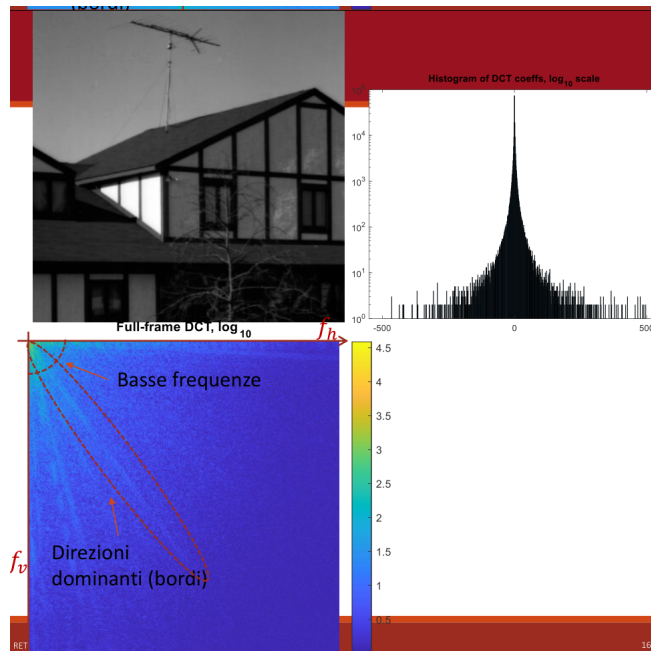


Figura 6.4: Trasformata coseno discreta (DCT) applicata alle righe ed alle colonne dell'immagine.

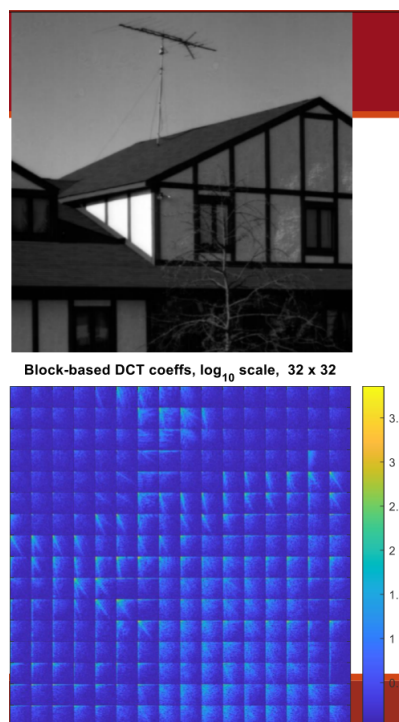


Figura 6.5: DCT applicata ai blocchi.

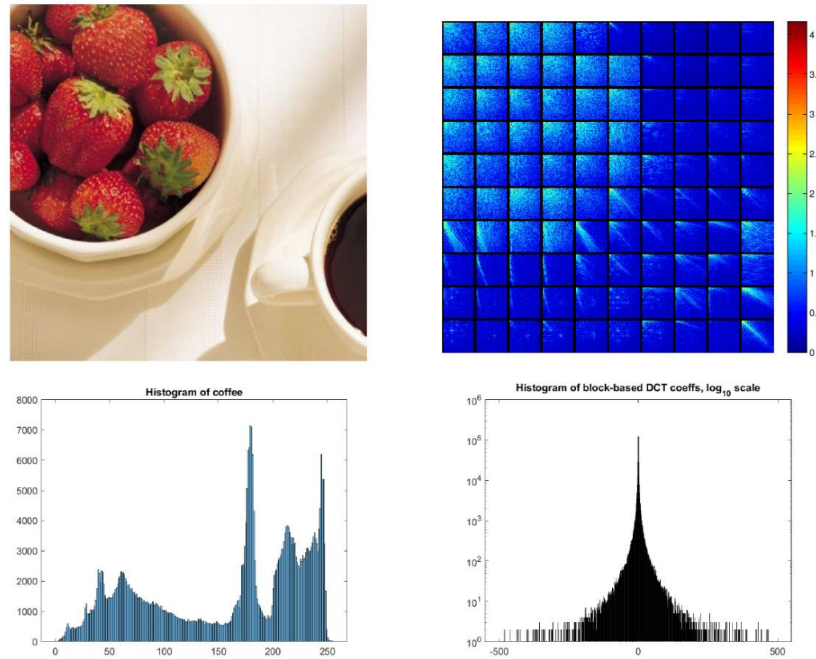


Figura 6.6: DCT applicata ai blocchi.

6.3 JPEG baseline

Lo standard JPEG è il più comune formato per le immagini digitali. Permette di rappresentare immagini a colori (3 canali) o a livelli di grigio (singolo canale). Dato che ogni canale viene trattato allo stesso modo, nelle successive sottosezioni andremo ad analizzare il modo in cui JPEG opera su un singolo canale.

6.3.1 Lo schema di codifica JPEG

Un'immagine x (più precisamente una singolo canale, quindi una matrice di valori tra 0 e 255) viene *centrata*, ovvero si sottrae 128 ad ogni valore; successivamente viene divisa in blocchi 8×8 , ognuno dei quali viene sottoposto a **DCT**, **quantizzazione** e **codifica lossless**.

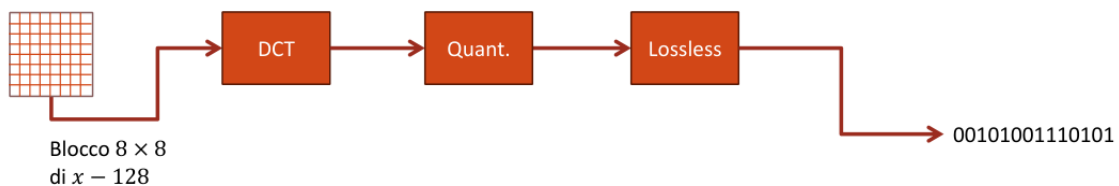


Figura 6.7: Schema di codifica JPEG.

6.3.2 Quantizzazione in JPEG

In JPEG viene utilizzata la quantizzazione uniforme (indicata con QU). La QU trasforma un numero reale x in un intero, detto indice di quantizzazione: $x_{\text{ind}} = Q(x, \Delta) = \text{round}(x/\Delta)$. Il valore x_{ind} verrà poi utilizzato in decodifica per ricostruire il valore di x , più precisamente, il valore ricostruito è $\tilde{x} = \Delta x_{\text{ind}} = \Delta \cdot \text{round}(x/\Delta)$, ovvero il multiplo intero di Δ più vicino a x .

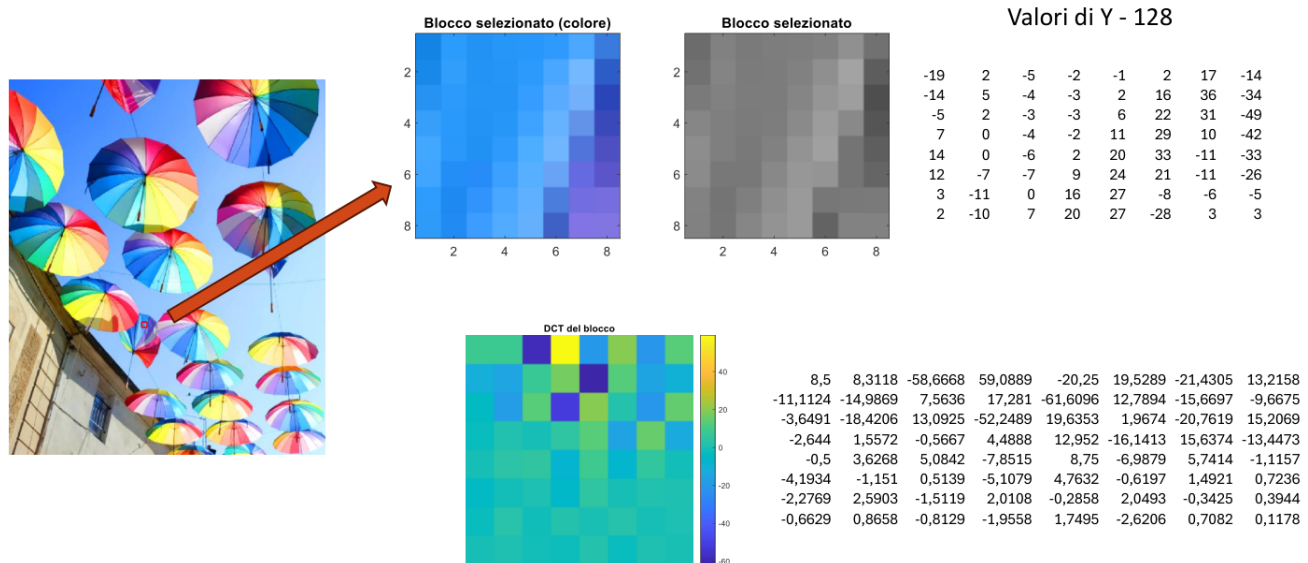


Figura 6.8: Esempio di blocco e coefficienti DCT.

Dunque, avere un Δ grande implica effettuare una "forte" quantizzazione e ciò comporta un tasso di codifica (numero di bit necessari per codificare il valore quantizzato) piccolo; valori piccoli comportano una migliore qualità di ricostruzione ma un tasso richiesto più grande.

JPEG permette di definire un valore diverso del passo di quantizzazione per ogni frequenza spaziale. Viene definita una **tabella di quantizzazione** $q(i,j)$, in cui ogni elemento indica il valore Δ usato per quantizzare il coefficiente a frequenza (i,j) . JPEG **non definisce** la tabella di quantizzazione, questa viene scelta dal codificatore ed inserita nel file codificato. Essa è necessaria al decodificatore per ricostruire i valori quantizzati a partire dagli indici interi. Ma come si sceglie la tabella? Sia per la *crominanza* che per la *luminanza*, sono state determinate per via sperimentale tramite test soggettivi (studi sulla qualità percepita).

$$q^* =$$

16	11	10	16	24	40	51	61
12	12	14	19	26	58	60	55
14	13	16	24	40	57	69	56
14	17	22	29	51	87	81	61
18	22	37	56	68	109	103	77
24	35	55	64	81	104	111	90
49	63	78	87	101	121	120	100
72	92	95	98	112	100	103	99

Figura 6.9: Tabella suggerita dallo standard per la luminanza. I valori in alto a sinistra sono più piccoli, ciò è dovuto al fatto che la maggior parte dell'informazione di un'immagine si trova alle basse e medie frequenze.

La tabella riportata produce un'immagine di **qualità** intermedia. Per poter ottenere una qualità maggiore si applica un **fattore di scala** alla tabella. L'utente può definire un *fattore di qualità* Q compreso tra 1 e 100 (a volte indicato in percentuale). Quando $Q = 1\%$, la matrice viene moltiplicata per un fattore di scala grande (50); quando $Q = 100\%$, il fattore di scala è piccolo (0.01). La matrice viene rappresentata comunque con valori tra 1 e 256 per poter essere rappresentata su 64 byte.

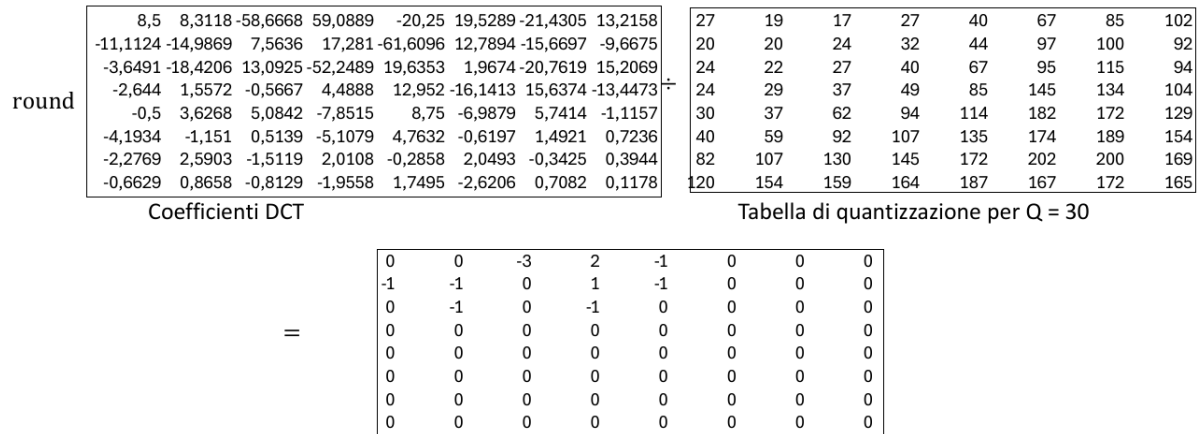


Figura 6.10: Esempio con fattore di qualità qualità $Q = 30$. Si nota che molti degli indici di quantizzazione sono nulli, in questo modo rimangono pochi valori da codificare in lossless.

Si nota che molto spesso i valori non nulli degli indici di quantizzazione sono concentrati nell'angolo in alto a sinistra del blocco (basse e medie frequenze). Per rendere più efficiente la codifica dei coefficienti, essi sono ordinati seguendo il **zig-zag scan**. Nell'esempio di Figura 6.10, lo zig-zag scan produce

$$\underbrace{0}_{\text{DC}}, \underbrace{0, -1, 0, -1, -3, 2, 0, -1, 0, 0, 0, 0, 1, -1, 0, -1, -1}_{\text{AC}}, \text{EOB}$$

dove: EOB (*End Of Block*) viene utilizzato per indicare che non ci sono più coefficienti non nulli; il primo coefficiente è detto AC, gli altri DC.

6.3.3 Codifica lossless coefficiente DC: codifica per categoria ed ampiezza

Il coefficiente DC è codificato in due passaggi: **predizione**; **codifica categoria/valore**. La predizione consiste nell'usare il precedente coefficiente DC come predittore del coefficiente corrente. Nell'esempio che stiamo considerando, il blocco precedente aveva come valore DC 1, quindi dobbiamo codificare l'errore di predizione:

$$e = x_{DC} - p = 0 - 1 = -1.$$

Si tratta di codificare -1 secondo la codifica per categoria ed ampiezza che abbiamo introdotto nella sottosezione 5.6.2.

- La categoria di -1 è $k = \lceil \log_2(|-1| + 1) \rceil = 1$. La categoria 1 è codificata con 010.
- $|-1| = 1$ è codificato in binario su k bit con 1; siccome $-1 < 0$ si considera il complemento di 1 che è 0.
- Infine il coefficiente DC è codificato con 0100.

6.3.4 Codifica lossless a coefficienti AC

Anche per la codifica dei coefficienti AC ci sono due passaggi: **codifica Run-Lenght**; **codifica per Run-categoria/valore**. La codifica Run-Lenght:

- consiste nell'indicare quanti zeri ci sono prima del prossimo coefficiente non nullo;

- successivamente si crea un nuovo simbolo (R, C) dove R è il numero di zeri e C è il valore del prossimo coefficiente quantizzato non nullo.
 - Se $R \geq 16$ si utilizza il valore speciale $(15, 0)$ per indicare che ci sono "più di quindici zeri" e seguito dal simbolo (R', C) , con $R' = R - 16$, che viene codificato seguendo le stesse regole.
 - Se non ci sono ulteriori valori non nulli, si usa il simbolo speciale $(0, 0)$ che significa *fine del blocco*.

Nel nostro esempio:

0, 0, -1, 0, -1, -3, 2, 0, -1, 0, 0, 0, 0, 1, -1, 0, -1, -1, EOB

↓

(1, -1) (1, -1) (0, -3) (0, 2) (1, -1) (4, 1) (0, -1) (1, -1) (0, -1) (0, 0)

I simboli (R, C) vengono codificati in due passaggi:

1. Prima si rappresenta C tramite categoria k e valore (come nel caso DC): si scrive $|C|_2$ su k bit e se $C < 0$ si calcola il complemento a 2 bit per bit.
2. Poi si codifica la coppia (R, k) con una codifica a prefisso.

Il codice per la categoria (DC) e quello per le coppie run/categoria, non sono standardizzati e quindi devono essere inseriti nel file JPEG.

R/C	Code	R/C	Code	R/C	Code
0/0 (EOB)	1010	2/1	11100	4/1	111011
0/1	00	2/2	11111001	4/2	1111111000
0/2	01	2/3	1111110111	4/3	1111111110010110
0/3	100	2/4	111111110100	4/4	11111111110010111
0/4	1011	2/5	1111111110001001	4/5	11111111110011000
0/5	11010	2/6	1111111110001010	4/6	11111111110011001
0/6	1111000	2/7	1111111110001011	4/7	11111111110011010
0/7	11111000	2/8	1111111110001100	4/8	11111111110011011
0/8	1111110110	2/9	1111111110001101	4/9	11111111110011100
0/9	111111110000010	2/10	1111111110001110	4/10	11111111110011101
0/10	1111111110000011	3/1	111010	5/1	1111010
1/1	1100	3/2	111110111	5/2	11111110111
1/2	11011	3/3	111111110101	5/3	1111111110011110
1/3	1111001	3/4	1111111110001111	5/4	11111111110011111
1/4	111110110	3/5	1111111110010000	5/5	11111111110100000
1/5	11111110110	3/6	1111111110010001	5/6	11111111110100001
1/6	1111111110000100	3/7	1111111110010010	5/7	11111111110100010
1/7	1111111110000101	3/8	1111111110010011	5/8	11111111110100011
1/8	1111111110000110	3/9	1111111110010100	5/9	11111111110100100
1/9	1111111110000111	3/10	1111111110010101	5/10	11111111110100101
1/10	1111111110001000				

Figura 6.11: Esempio di tabella R, k .

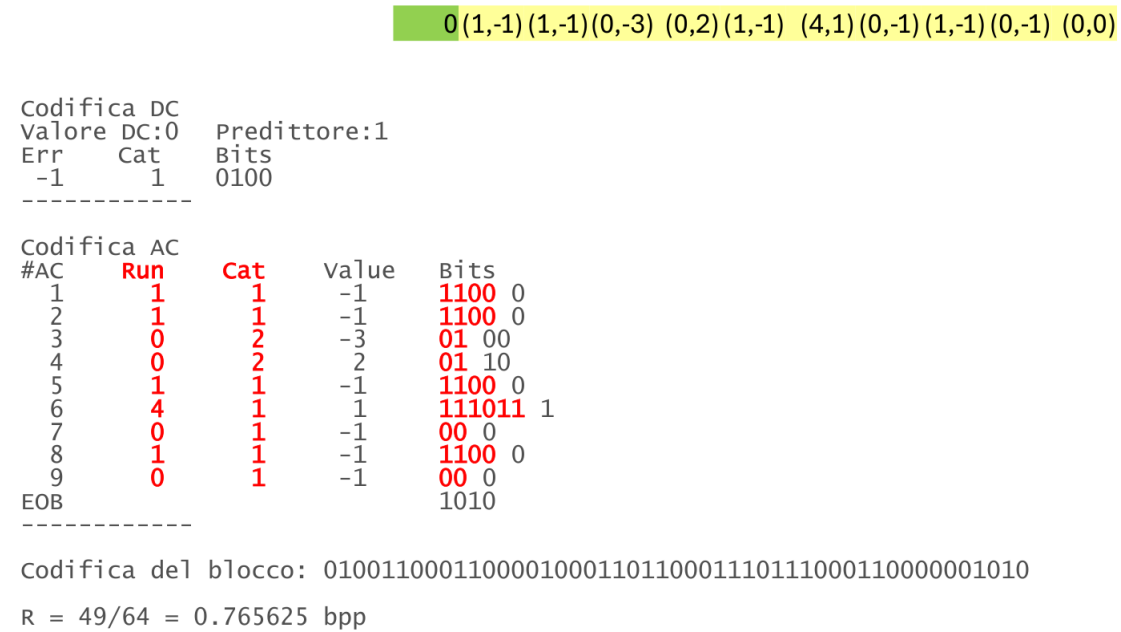
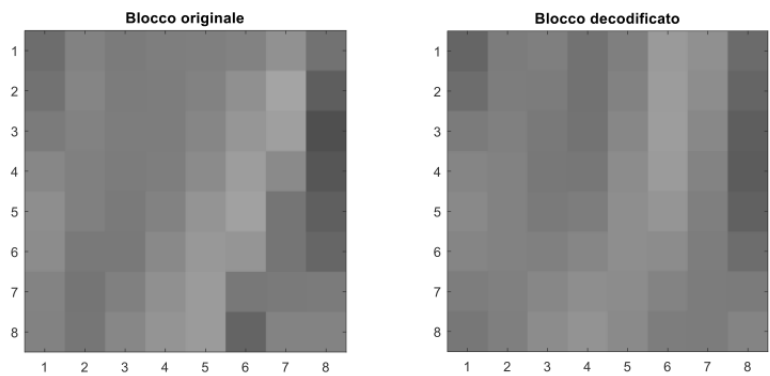


Figura 6.12: Esempio di codifica lossless di un blocco.



Capitolo 7

La compressione video: metodologie e standard

7.1 Principi di base

La compressione video è necessaria in quanto i formati video non compressi (vedi sezione 3.4) richiedono tassi di trasmissione molto elevati. In generale la compressione video sfrutta la ridondanza spaziale delle immagini (come quella vista in JPEG), ma il grosso della riduzione di dimensione del video deriva dalla riduzione della ridondanza temporale (infatti le immagini consecutive di un video sono molto simili le une alle altre).

I principali parametri di progetto di un codificatore video riguardano i seguenti punti:

- la modalità di predizione temporale (parametri della **stima del movimento**)
- la struttura del *Group Of Pictures*;
- il modo in cui si decide come codificare ogni singolo blocco di pixel (sono disponibili numerose alternative).

7.2 La predizione temporale tramite stima e compensazione del movimento

Un semplice modello del segnale video consiste nel considerare ogni nuova immagine come risultante del movimento degli oggetti e della camera. Così facendo, l'immagine corrente può essere **predetta** spostando opportunamente gli oggetti dell'immagine precedente. L'errore di predizione (dato dalla vera immagine meno la predizione) è tipicamente un segnale molto sparso che può essere quindi codificato in modo efficace.

La tecnica più comune di predizione temporale è la stima e compensazione del movimento.

- La **stima del movimento** consiste nell'associare ad ogni pixel $p = (x, y)$ dell'immagine corrente f_n un **vettore di movimento** $v(x, y)$ che punta verso un pixel dell'immagine di riferimento f_{n-N} .
- La **compensazione del movimento** consiste nel creare una predizione dell'immagine corrente: $\hat{f}_n(p) = f_{n-N}(p + v(p))$.

L'approccio più comune è quello di cercare il movimento tramite **block-matching** (Figura 7.1), dove si considerano i blocchi di pixel e non i singoli pixel. La notazione $B_k^{(p)}$ indica un blocco di pixel centrati nella posizione $p = (x, y)$ della k -esima immagine. Un tale blocco può essere visto come un insieme di $N \times M$ valori, quindi un elemento di $\mathbb{R}^n$ (con $n = NM$).

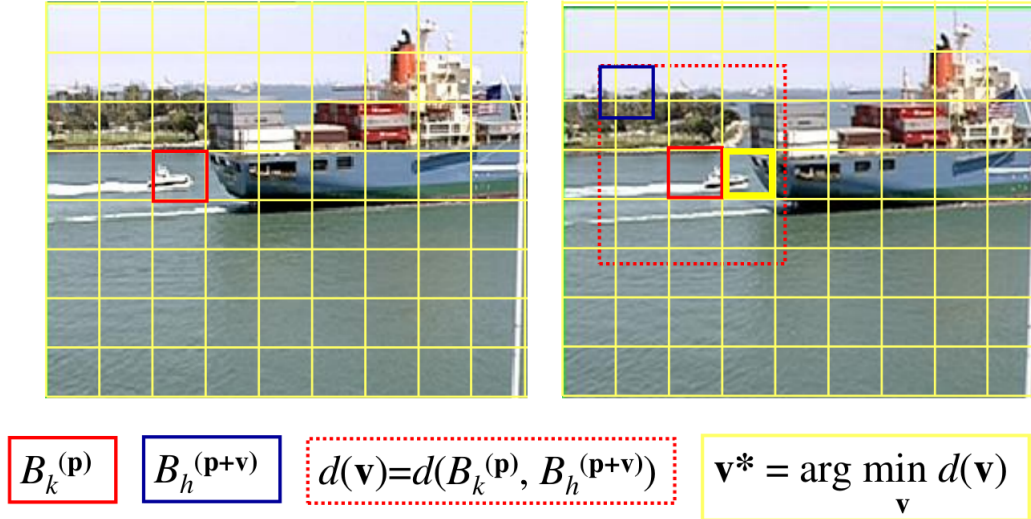


Figura 7.1: Stima del movimento per block-matching.

Nella Figura 7.1 cerchiamo un blocco dell'immagine h che somigli a $B_k^{(p)}$. Il blocco cercato $B_h^{(p+v)}$ è spostato rispetto alla posizione p tramite il vettore di traslazione v . Per trovare il blocco consideriamo la minima distanza tra

$$d(v) = d(B_k^{(p)}, B_h^{(p+v)})$$

questo ci fornisce un'indicazione su quanto i due blocchi sono simili. Il vettore stimato è quello che minimizza la distanza:

$$v^* = \underset{v \in W}{\operatorname{argmin}} d(v).$$

7.2.1 Parametri di progetto della stima del movimento

I parametri da considerare sono:

- **Forma e dimensione blocchi:** blocchi piccoli permettono di seguire più fedelmente il movimento, ma implicano più vettori, quindi più bit di codifica e più calcoli.
- Insieme di vettori candidati W , detto anche **finestra di ricerca**: aumentando le dimensioni della finestra, si aumenta il numero di possibili predittori, in questo modo aumenta la qualità della predizione ma anche il costo computazionale.
- La funzione di **distanza** d : tipicamente si utilizza una variante dell'MSE, detta SAS (*Sum Of Absolute Values*) che è molto più rapida.
- **Strategia** di ricerca: si possono testare tutti i vettori della finestra (full-search) oppure scegliere delle strategie intelligenti che riducono la complessità con minimo impatto sulla qualità.



Figura 7.2: Stima del movimento: immagine di riferimento e immagine corrente.

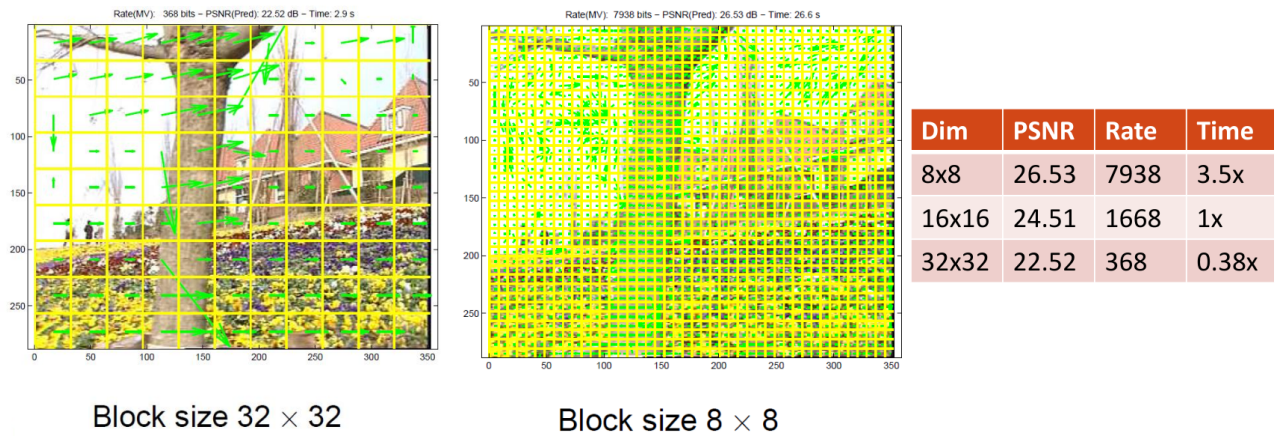


Figura 7.3: Effetto della dimensione dei blocchi.

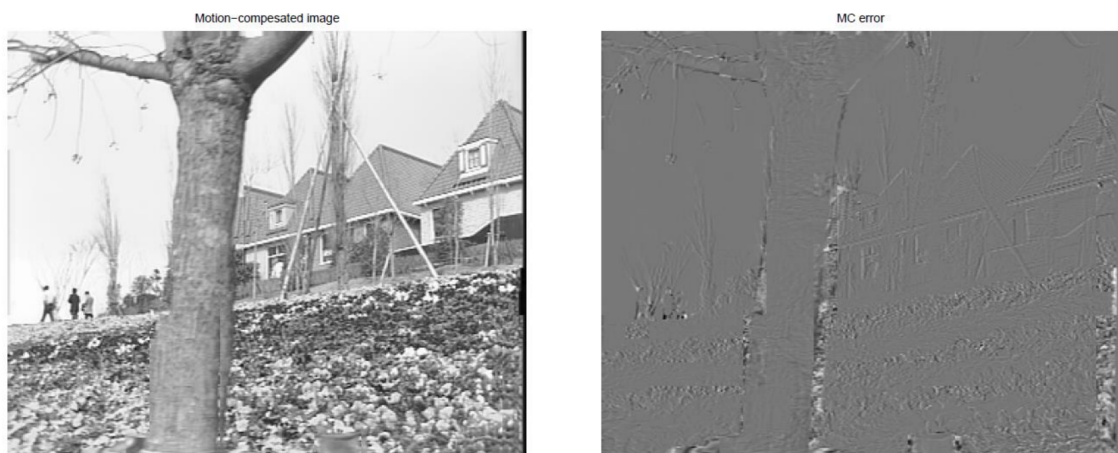


Figura 7.4: Esempio compensazione del movimento. (a) Immagine compensata in movimento: ogni blocco è rappresentato da suo predittore. (b) Errore di predizione: differenza tra l'immagine compensata e la vera immagine: è un segnale sparso quasi ovunque.

7.2.2 Considerazioni sulla stima del movimento

La stima del movimento serve per trovare il miglior predittore del blocco corrente, non per individuare il movimento fisico degli oggetti (spesso le due cose coincidono ma non è sempre così). L'errore di predizione $e = B_k^{(p)} - B_h^{(p+v^*)}$ viene codificato con un metodo simile a JPEG. L'errore generalmente è più sparso del blocco $B_k^{(p)}$ e quindi la codifica è potenzialmente più efficace. Se non si riesce a trovare un buon predittore (es: quando un nuovo oggetto entra nella scena), è meglio non usare la predizione temporale.

7.3 I tipi di immagine ed il group of picture (GOP)

Limitare l'estensione delle tecniche predittive è molto importante per poter limitare la propagazione degli errori, per rendere possibile l'accesso casuale al video e per limitare la complessità computazionale del codificatore. A tali scopi si utilizzano i **tipi d'immagine**: limitano il modo in cui ogni blocco di pixel può essere codificato.

7.3.1 I tipi di immagine

I tipi d'immagine vengono organizzati in una struttura periodica che viene chiamata group of pictures. Consideriamo tre tipi di immagine (*frame*):

- **Frame I** → Intra coded
- **Frame B** → Predictive
- **Frame P** → Bi-prediction

I frame I e P fanno parte degli *Anchor Frame* (AF). Per i frame I non è possibile utilizzare la predizione temporale ma eventualmente la predizione spaziale dalla stessa immagine; nei frame P, ogni blocco è da un singolo blocco dell'immagine di riferimento, oppure si può usare la predizione spaziale; infine nei frame B, ogni blocco può essere predetto da 2 blocchi (oppure da uno, oppure con predizione spaziale).

7.3.2 Il Group Of Pictures

I frame vengono organizzati in Group Of Pictures. La struttura di un gruppo:

- ogni gruppo inizia con un frame I;
- indichiamo con N l'intervallo tra due I (8 nell'immagine);
- indichiamo con M l'intervallo tra due AF (2 nell'immagine).

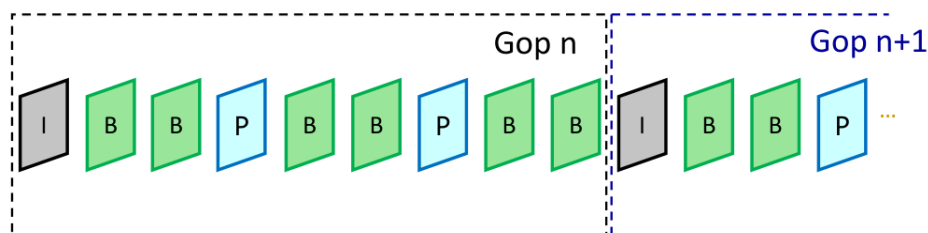


Figura 7.5: Struttura del GOP.

I frame I Per cominciare la codifica di un video, il primo frame non può avere predizione temporale: deve essere codificato in modo *Intra*. La codifica Intra è simile a JPEG, ma può sfruttare anche la predizione spaziale. I vantaggi del frame intra sono:

- bassa complessità (niente stima del movimento);
- punto di accesso in decodifica (accesso casuale);
- limite alla propagazione degli errori.

Lo svantaggio è che, a parità di qualità, richiedono molti più bit di codifica rispetto alle immagini P e B.

I frame P Ogni blocco di un frame P può essere codificato in modalità Intra ma anche in modalità INter, cioè con predizione temporale. Si utilizza una singola immagine come riferimento per la predizione, per esempio l'Anchor Frame precedente. Il vantaggio è che sono molto più compressi dei frame Intra, lo svantaggio è l'elevata complessità a causa della stima del movimento.

I frame B I frame B possono essere predetti da due immagini diverse, tipicamente una immagine passata ed un'immagine futura: in questo modo aumenta la probabilità di trovare predittori efficaci per ogni blocco; l'ordine di codifica non coincide con l'ordine di visualizzazione. Il vantaggio dei frame B è che permettono un altro livello di compressione, lo svantaggio è la complessità elevata (2 stime del movimento).

Capitolo 8

Indici di prestazione di rete

8.1 Principi di base

8.1.1 Il servizio di trasporto

Le applicazioni multimediali sfruttano un preciso servizio del livello di trasporto: il canale **per flussi di bit, end-to-end** e **per processi**. Più precisamente:

- canale per **flusso di bit**: a sorgente non deve preoccuparsi della lunghezza del flusso prodotto, anche se può dare indicazioni su come frammentarlo;
- **end-to-end**: ovvero da sorgente a destinazione. L'applicazione multimediale non deve e non può occuparsi di definire il percorso seguito dal messaggio trasmesso, ma solo di definire chi sono il trasmettitore e il ricevitore;
- **per processi**: significa identificare un particolare processo su di un particolare host come sorgente, ed analogamente, un particolare processo su un particolare host come destinatario.

8.1.2 La pacchettizzazione

Il servizio di trasporto tipicamente divide il flusso di bit in pacchetti. Le informazioni multimediali non vengono trasmesse in modo monolitico ma spezzettate in unità di dati (chiamati **pacchetti**). Secondo il principio dell'architettura di rete stratificata, l'applicazione multimediale non dovrebbe essere a conoscenza di questa pacchettizzazione. Tuttavia, in alcuni casi, **può essere utile che l'applicazione multimediale dia indicazioni** su come suddividere il bit stream in pacchetti.

La pacchettizzazione di un flusso di bit $s_b(n)$ consiste nel suddividerlo in blocchi (*source data blocks*) detti appunto, pacchetti. La rete poi si occupa di trasmettere un blocco alla volta. Per fare questo, è necessario trasmettere anche delle informazioni di controllo (**header**), per esempio dei codici identificativi del blocco, della sorgente, del destinatario, etc.

I vantaggi della trasmissione a pacchetti rispetto ad una trasmissione in blocco sono diversi:

1. **efficienza**: la suddivisione in pacchetti permette di utilizzare la rete in modo più efficiente dato che i pacchetti possono essere instradati attraverso percorsi diversi;
2. **affidabilità**: in caso di errori di trasmissione, solo i pacchetti danneggiati devono essere ritrasmessi e non tutto l'intero messaggio;
3. **scalabilità**: reti basate sulla pacchettizzazione possono crescere e gestire un numero maggiore di dispositivi senza bisogno di cambiamenti nell'infrastruttura;

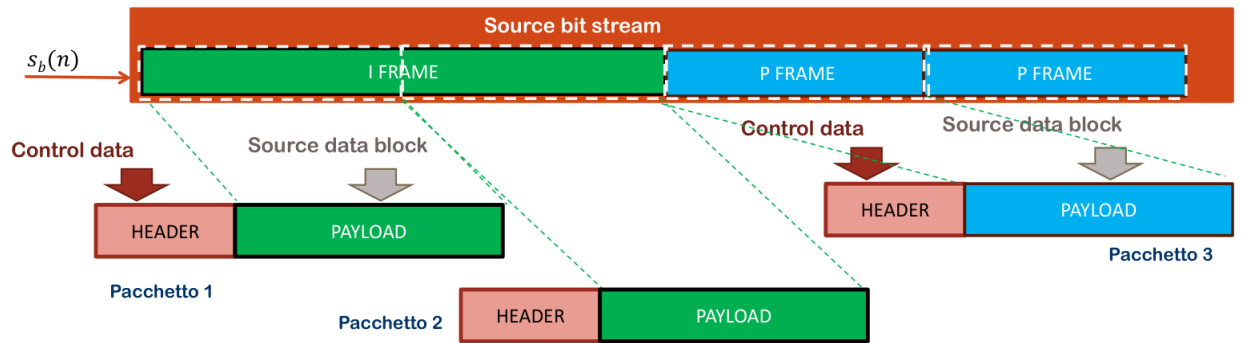


Figura 8.1: Pacchettizzazione di un flusso di bit.

4. **flessibilità**: facilita il supporto a tipi di dato diversi su di una stessa rete.

Riassumendo: la pacchettizzazione dei dati è fondamentale per garantire che le reti di calcolatori siano affidabili, efficienti e pronte a supportare le esigenze di comunicazione moderne.

8.1.3 Paradigma di trasmissione

Le applicazioni producono dati ad una determinata velocità (non necessariamente costante). Questi dati passano attraverso il livello di trasporto il quale li suddivide in unità più piccole e aggiunge le informazioni di protocollo (header). I pacchetti formati al livello 4 passano successivamente al livello 3. Ogni livello può spezzare le unità di dati ricevute e aggiungere ulteriori intestazioni.

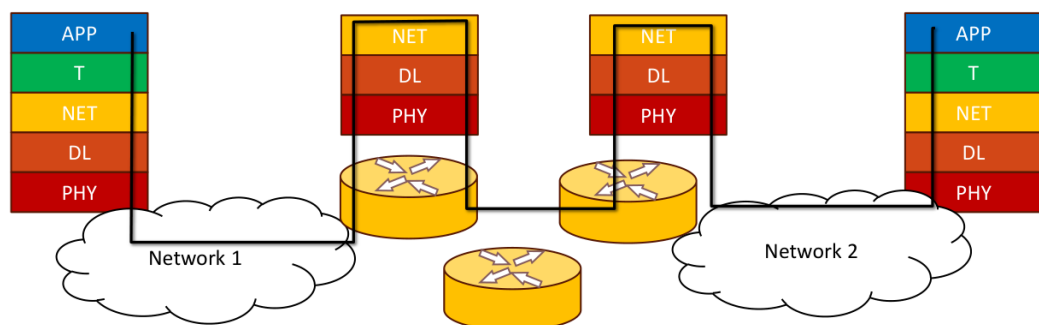


Figura 8.2

Nella fase di ricezione avviene il processo inverso e in questo modo vengono ricostruiti i dati dell'applicazione.

8.1.4 Indici di prestazione di rete

I parametri principali da considerare sono i seguenti:

- il **bit-rate**: numero di bit al secondo che si possono trasmettere;
- il **ritardo medio**: quanto tempo passa in media tra l'istante in cui un pacchetto viene prodotto al trasmettitore e l'istante in cui esso è completamente ricevuto;
- la varianza del ritardo, o **jitter**;

- il **prodotto banda-ritardo**;

Esistono altri indici che descrivono l'affidabilità della trasmissione:

- la probabilità di errore sul bit (o **bit error-rate**);
- la probabilità di perdere un pacchetto;
- la garanzia (o assenza di garanzia) dell'ordine di ricezione dei pacchetti.

8.2 Metriche per il traffico

L'unità di misura che utilizzeremo è il *bit/s*, ovvero i bit trasmessi/ricevuti in un secondo. Non utilizzeremo la notazione binaria (secondo la quale $1\text{MByte} = 2^{20}\text{byte}$).

8.2.1 Bitrate, larghezza di banda (bandwidth), Throughput, Goodput

Introduciamo:

- Indichiamo il **link bitrate** con R_0 [*bit/s*]: si tratta della velocità massima di trasmissione a livello di bit tra nodi adiacenti. In genere viene utilizzata per indicare la velocità di trasmissione dati offerta dal livello PHY al livello superiore L2.
- Indichiamo la larghezza di banda¹ o **bandwidth** con W [*Hz*].

8.2.2 Throughput e Goodput

Bitrate, Throughput e Goodput sono concetti molto simili: si riferiscono tutti a una velocità di trasferimento tra due entità peer in una comunicazione; la differenza sta nel layer considerato. In genere, il bit-rate si riferisce ai livelli inferiori (PHY, MAC) e il Throughput e il Goodput ai livelli superiori.

Il **throughput** viene indicato con S [*bit/s*]. Indica la velocità media di trasferimento dati dal punto di vista del livello superiore. Il **vincolo fondamentale** è che $S \leq R_0$. Gli strati superiori non possono essere più veloci di quelli inferiori a causa di: presenza di overhead; possibili congestioni; tempi di inattività forzati introdotti dai protocolli. Il throughput viene calcolato il rapporto tra la quantità di dati trasmessi ed il tempo impiegato per farlo.

Possiamo definire la seguenti metriche di throughput:

- *throughput medio*

$$s_n(t; T) = \frac{b_n(t) - b_n(t - T)}{T}$$

dove $b_n(t)$ è il numero cumulativo di bit trasmessi dall'istante iniziale fino all'istante t , visto da layer n . Al numeratore troviamo il numero di bit ricevuti nell'intervallo T .

- *throughput istantaneo*: limite superiore del throughput medio per $T \rightarrow 0$, quindi è la derivata di $b_n(t)$

$$s_n(t) = \frac{\partial b_n(t)}{\partial t}.$$

- *throughput di lungo termine o a regime stazionario o asintotico*:

$$S_n = \lim_{T \rightarrow \infty} s_n(t; T).$$

Il throughput a lungo termine a livello di applicazione è definito **goodput**.

¹Il termine "larghezza di banda" viene talvolta (impropriamente) utilizzato al posto di bitrate, nel qual caso viene espresso in *bit/s*.

8.2.3 PDU, SDU, headers

Due entità dello stesso livello (*entità peer*) si scambiano unità di dati di protocollo (**PDU, protocol data unit**). Per esempio, una PDU di un'applicazione di comunicazione video (L5) potrebbe essere un frame intero. La PDU viene trasmessa al livello sottostante, che la tratta come un'unità di dati di servizio (**SDU, service data unit**). I livelli sottostanti aggiungono le proprie informazioni di controllo del protocollo (**PCI, protocollo control unit, header**), necessarie per implementare il proprio protocollo di comunicazione e creare la nuova PDU, che viene poi trasmessa al livello inferiore o al prossimo nodo.

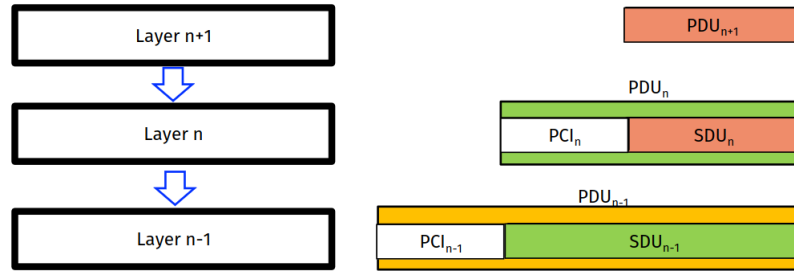


Figura 8.3: PDU, SDU e PCI.

Come impatta sulla velocità del livello l'aggiunta dei dati di controllo? Il throughput massimo che il livello n può offrire al suo livello superiore è

$$S_{n+1} = S_n \cdot \frac{|SDU_n|}{|PDU_n|} = S_n \cdot \frac{|SDU_n|}{|PCI_n| + |SDU_n|}$$

dove i valori $|SDU_n|$, $|PDU_n|$, $|PCI_n|$ indicano il numero di bit rispettivamente di SDU , PDU e header al livello n . L'efficienza del protocollo di livello n è definita come segue:

$$\eta = \frac{S_{n+1}}{S_n} = \frac{|SDU_n|}{|PDU_n|} = \frac{|SDU_n|}{|PCI_n| + |SDU_n|} = \frac{|PDU_{n+1}|}{PDU_n}, \quad 0 \leq \eta \leq 1.$$

Notiamo che l'efficienza aumenta con l'aumentare delle dimensioni della SDU e diminuisce con il diminuire dell'header. Tuttavia esistono dei vincoli che impediscono di avere valori di SDU troppo grandi o header troppo piccoli.

Consideriamo un esempio: poniamo che il layer $n+1$ crei un PDU di $1200B$; il throughput a livello $S_n = 100Mbps$; l'header a livello n ha dimensione $60B$. Troviamo il throughput massimo a livello a $n+1$:

$$S_{n+1} = 10^8 \frac{1200}{1200 + 60} \approx 95.24Mbps$$

8.3 Metriche di ritardo

8.3.1 Ritardo end-to-end, ritardo point-to-point e RTT

Definiamo:

- Il **ritardo punto-punto** d_{p2p} è il tempo che intercorre tra l'inizio della trasmissione di un pacchetto in un nodo e la ricezione completa del pacchetto del nodo successivo.
- Il **ritardo end-to-end** d_{e2e} è il tempo che intercorre tra l'inizio della trasmissione di un pacchetto in un nodo e il momento in cui il pacchetto viene ricevuto completamente

alla destinazione finale. d_{e2e} è dato dalla somma di tutti i ritardi nodali tra sorgente e destinazione. Ponendo che un collegamento sia composto da K link, allora

$$d_{e2e} = \sum_{i=1}^K d_{\text{nodale},i}$$

- Il **round-trip time (RTT)** (o **ping**) indica il tempo di andata e ritorno tra due nodi A e B di una rete. Il valore è dato dalla somma dei ritardi end-to-end da A a B e da B ad A (si può approssimare come $RTT = 2d_{e2e}$).

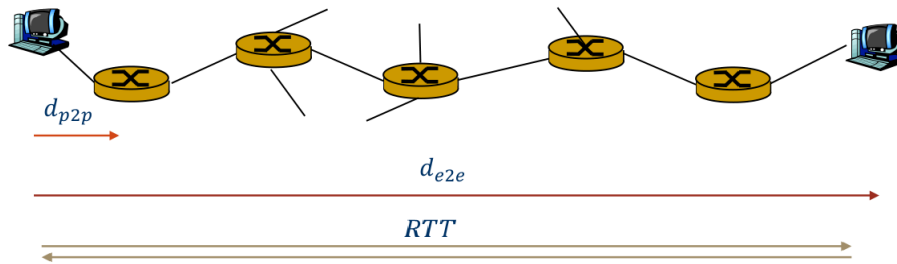


Figura 8.4: Ritardo punto-punto e ritardo end-to-end.

8.3.2 Ritardo nodale

Il ritardo nodale è dato da

$$d_{\text{nodale}} = d_{\text{proc}} + d_{\text{queue}} + d_{\text{trans}} + d_{\text{prop}}$$

dove:

- d_{proc} è il **ritardo di processing**. Nelle operazioni di processing vengono considerate le seguenti operazioni:
 - controllo di errore;
 - determinare il link di uscita (instradamento del pacchetto);

Tipicamente il valore è molto piccolo e può essere ridotto migliorando i meccanismi di instradamento.

- d_{queue} è il **ritardo di accodamento**. Si tratta del tempo che il messaggio trascorre in un buffer, in attesa di essere elaborato. Dipende dal traffico della rete e dalle caratteristiche strutturali.
- d_{trans} è il **ritardo di trasmissione**. Il valore è dato da L/R dove R è il bitrate e L è la lunghezza del pacchetto.
- d_{prop} è il **ritardo di propagazione**. Il valore è dato da d/c dove d è la lunghezza del collegamento fisico e c è la velocità di propagazione del mezzo.

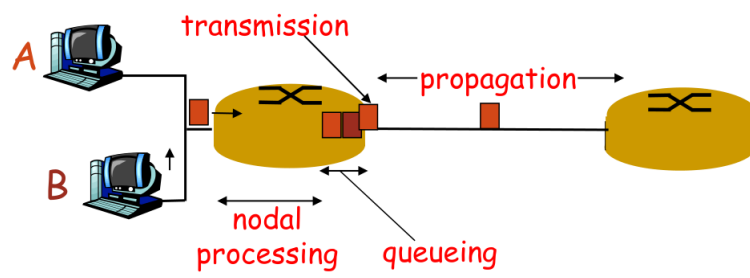


Figura 8.5: Ritardo nodale.